Федерация спортивного программирования России · спецтрек

FSP Talent

Цифровая платформа подбора ИТ-специалистов с обратной механикой и верифицированным профилем достижений участника ФСП

Сопроводительная документация · 08.10.2026

Содержание

1. Обзор решения
2. Уточнения заказчика
3. Архитектура
4. Механика подбора
5. Механика тестирования
6. Интеграция с реестром ФСП
7. API
8. Сборка и развёртывание
9. Интерфейс

Документ собран из исходных файлов репозитория (`README.md` , `docs/*.md`). Отчёты валидации — `evaluation/results/` , API — Swagger UI по адресу `/api/docs` .

1. Обзор решения

Цифровая платформа с **обратной механикой подбора**: кандидат проходит тест на заявленный грейд и получает объективную категорию («Backend · Middle»), профиль автоматически обогащается достижениями Федерации спортивного программирования, а **работодатель сам** описывает потребность, получает ранжированную и объяснимую подборку и направляет конкретному человеку приглашение с зарплатной вилкой.

> MVP для спецтрека ФСП «Цифровая платформа-агрегатор ИТ-вакансий с верифицированным профилем достижений участника». Всё ниже работает «из коробки» на демо-данных.
>
> **Уточнения заказчика** (ответы на вопросы команд) и что по каждому сделано — в docs/clarifications.md: адресные регулярные задания с автопроверкой и рубрикой, несколько категорий, статус «грейд не подтверждён», проверка честности теста по поведению, тест-ретест всей процедуры и анализ разделяющей силы каждого задания, профильность дисциплин ФСП.

Русский и английский. Интерфейс, ответы сервера (ошибки, объяснения подбора, рекомендации по тесту, справочники, письма) и весь банк заданий доступны на двух языках. Язык определяется по браузеру, переключается RU / EN на любом экране и сохраняется в учётной записи. Тест можно проходить на любом языке и переключать по ходу — варианты ответов и калибровка общие (docs/architecture.md).

Быстрый старт

```
docker compose up --build
```

Приложение	**http://localhost:8080**
API (Swagger)	http://localhost:8000/api/docs
Почта (коды подтверждения)	http://localhost:8025

Без Docker — см. docs/deployment.md (Python 3.12 + Node 20, SQLite по умолчанию).

Демо-аккаунты (пароль у всех: `demo12345`)

Роль	E-mail	Что показывает
Кандидат	`candidate@demo.ru`	Backend · Middle + DevOps · Junior, 3 достижения ФСП, приглашения, предложенное задание
Кандидат	`fresh@demo.ru`	Профиль заполнен, тест не пройден — можно сразу пройти тестирование
Работодатель	`hr@demo.ru`	«Технологии Будущего»: 3 потребности, подборки, приглашения, задания с решениями на проверку
Работодатели	`jobs@severcode.demo`, `team@datalab.demo`, `hr@oblakotech.demo`, `people@mediaplatform.demo`	Другие компании
Кандидаты	`cand01@talent.demo` … `cand48@talent.demo`	48 синтетических кандидатов

На главной странице есть кнопки входа в демо-аккаунты. Для живой регистрации с ФСП: зарегистрируйтесь с e-mail `olymp@demo.ru` (достижения подтянутся автоматически) или привяжите свободный ФСП ID `FSP-200001` / `FSP-200002` / `FSP-200003` (код подтверждения показывается в демо-режиме).

Сквозной сценарий демонстрации

1. **Кандидат**: «Регистрация» → код подтверждения (в демо виден на экране и в Mailpit) → профиль (можно загрузить PDF-резюме) → «Тестирование»: опрос — Backend, Middle → «Начать тест» → 20 вопросов → результат: грейд, категория, шкальный балл, компетенции, уверенность → «Достижения ФСП» → «Настройки»: включить публикацию профиля.
2. **Работодатель** `hr@demo.ru`: «Новая потребность» → описать текстом («Middle Python, FastAPI, PostgreSQL, Docker, 180–250 тыс., удалёнка») → «Разобрать описание» → «Получить подборку» → рекомендованные категории и рейтинг с объяснением → «Профиль и объяснение» (вклад факторов, контакты скрыты) → «Пригласить» (вилка обязательна).
3. **Кандидат**: «Приглашения» → компания, предложение, **зарплата до ответа**, «почему выбрали вас» → «Принять» → у работодателя открываются e-mail, телефон, Telegram.
4. **Регулярные задания**: кандидат в «Коротких заданиях» видит задачу, которую система предложила именно ему («работодатель рассматривает вас для „Python Backend Developer“»), отвечает; работодатель в «Коротких заданиях» видит предварительную оценку по ключевым идеям и ставит оценку по рубрике — в подборке появляется «свежий сигнал».

Дополнительно: сравнение кандидатов «почему A выше B», банк кандидатов с фильтрами, публичные вакансии и отклики, несколько категорий у кандидата, жалоба на приглашение, PDF-профиль.

Видео сценария (≈6 мин, 6 глав: кандидат, работодатель, ответ кандидата, регулярные задания, отклик, два языка; субтитры объясняют механики тестирования и подбора) записывается на запущенном стенде: `pip install playwright imageio-ffmpeg && python -m playwright install ffmpeg`, затем `python scripts/record_demo.py` → `presentation/FSP_Talent_demo.mp4` (демо-данные сбрасываются до и после записи).

Ключевые механики

Подбор (docs/matching.md). Объяснимая формула из 9 компонентов: стек через граф навыков с учётом *доказательности* (подтверждён ли навык тестом), подтверждённый грейд, результат теста относительно требуемого уровня, специализация, ФСП (нет истории = нейтральная база, без штрафа; вес достижения зависит от профильности дисциплины), актуальность и «свежий сигнал» регулярных заданий, зарплата, опыт, формат + мягкое «И» для обязательных навыков. Каждый процент разложен на вклады факторов; есть сравнение A vs B; подборка сохраняется и уточняется фильтрами. У кандидата может быть до трёх категорий — в подборке его представляет категория специализации потребности.

Тестирование (docs/testing.md). Банк из 252 калиброванных заданий и 25 параметрических семейств (новые числа для каждого кандидата) по 4 специализациям и 21 компетенции. Индивидуальный вариант собирается по единому плану (blueprint) — одинаковые квоты компетенций и профиль сложности; оценка по IRT (3PL, EAP) делает результаты разных вариантов сопоставимыми. Детектор утечки (person-fit + разрыв «статические vs параметрические задания») и проверка поведения (уходы со вкладки, вставка, быстрые верные ответы на трудные для кандидата вопросы) требуют подтверждающей формы. Заявленный грейд без теста помечается «не подтверждён». Кулдауны: повтор через 30 дней, смена грейда — раз в 90 дней (у каждой категории свой срок), тест ниже — сразу после провала, тест выше — сразу после уверенного результата.

Регулярные задания (docs/testing.md §8). Система раз в неделю предлагает кандидату задачу от работодателя — по грейду, по потребности, где его уже рассматривают, по слабо подтверждённой компетенции. Автопроверка (число / текст) или подход с рубрикой работодателя (корректность, полнота, обоснование); предварительная оценка по ключевым идеям помогает разбирать ответы. Результат — свежий сигнал в подборке; грейд меняет только тест.

ФСП (docs/fsp-integration.md). Порт `FSPRegistry` с мок-адаптером и HTTP-адаптером, мок-API реестра, автопривязка по подтверждённому e-mail, ручная привязка ФСП ID с подтверждением владения, совместимость с Keycloak (OIDC-claims в JWT).

Валидация (воспроизводимо, без БД)

```
python evaluation/evaluate_tests.py      # механика тестирования
python evaluation/evaluate_matching.py   # бенчмарк подбора: P@K, R@K, MRR, NDCG
python evaluation/tune_matching.py       # подбор гиперпараметров с проверкой на отложенных данных
```

Тестирование (симуляция с известной истиной, 1 600 кандидатов)	
Точность грейда / взвешенная к / в пределах ±1	82,6% / 0,904 / 99,9%
Повторное прохождение всей процедуры одним профилем: тот же грейд / тот же или соседний / kw	75,9% / **99,8%** / 0,866
Задания, хорошо разделяющие слабых и сильных (индекс D ≥ 0,3) · приемлемо (0,2–0,3)	47% · 24%
Эквивалентность вариантов (SD ожид. балла): blueprint vs случайный отбор	0,013 vs 0,045
Junior выучил ответы 10 «слитых» вариантов Middle → доля сдавших: единый тест / наш подход	98% / **1,3%**
Ищет ответы на стороне → доля сдавших без проверки поведения / с ней	51,5% / **0,3%**
Ложные срабатывания: детектор утечки / проверка поведения	4,7% / 0%

Подбор (60 потребностей × 600 кандидатов)	P@5	NDCG@10	MRR
Поиск по ключевым словам резюме	0.377	0.461	0.655
Ключевые слова + заявленный грейд	0.700	0.692	0.817
FSP Talent	**0.880**	**0.808**	**0.933**

Отчёты: `evaluation/results/*.md`.

Архитектура

```
frontend/   React 19 + TypeScript + Vite; типы API генерируются из OpenAPI
  src/i18n/           словари RU / EN, плюрализация, форматирование, переключатель языка
  src/**/*.test.ts*   124 теста (Vitest + Testing Library)
backend/    FastAPI · SQLAlchemy 2 · Alembic · PostgreSQL (SQLite локально)
  app/api/            REST-роутеры (без бизнес-логики)
  app/services/       сценарии: auth, кандидат, категории, работодатель, приглашения, приватность, задания
    testing/          Testing Engine: банк → blueprint → генерация → IRT → правила грейдов → честность
    matching/         Matching Engine: признаки → скоринг → объяснение → снимок подборки
    fsp/              порт реестра ФСП, адаптеры, сила сигнала
    nlp/              разбор PDF-резюме и текстового описания потребности (RU / EN)
  app/i18n/           язык запроса (Accept-Language), каталоги сообщений RU / EN
  app/repositories/   доступ к данным
  app/models/         26 таблиц, миграции в alembic/
  app/domain/         справочники и банк заданий
  app/seed/           демо-данные (кандидаты проходят настоящий движок тестирования)
  tests/              446 тестов (pytest): движки, категории, задания, честность, приватность, локализация
  scripts/            экспорт OpenAPI, анализ заданий по реальным попыткам (item_analysis.py)
evaluation/  офлайн-валидация тестирования и подбора
docs/        архитектура, механики, интеграция ФСП, API, развёртывание
```

Подробнее: docs/architecture.md · API: docs/api.md · развёртывание: docs/deployment.md. Вся документация одним файлом — docs/FSP_Talent_documentation.pdf (собирается `scripts/build_docs_pdf.py`).

Проверки

```
cd backend && pytest -q && ruff check app tests scripts
cd frontend && npm test && npm run typecheck && npm run lint && npm run build
python scripts/smoke_test.py --base http://localhost:8080
```

Те же проверки (плюс применение и откат миграций и короткий бенчмарк подбора) запускаются в CI на каждый push и pull request: .github/workflows/ci.yml.

Ограничения MVP

- Реестр ФСП — синтетический мок (реальный API не предоставлен); интеграция заменяется адаптером.
- Пороги проверки поведения подобраны на модели времени ответа; на пилоте их стоит уточнить по реальным попыткам (скрипт анализа заданий и журнал проверок готовы).
- Параметры заданий — экспертная калибровка; процедура перекалибровки по ответам реализована и проверена на симуляции, но требует пилота на реальных кандидатах.
- Веса матчинга подобраны на синтетическом бенчмарке; после запуска — дообучение на исходах приглашений.
- Демо-режим показывает код подтверждения в интерфейсе (`EMAIL_DEV_EXPOSE_CODE=false` отключает).
- Пользовательские тексты (резюме, описания вакансий, названия соревнований ФСП) не переводятся. Английская версия заданий использует ту же калибровку — на пилоте стоит проверить эквивалентность заданий между языками (DIF-анализ).
- Защита от недобросовестных работодателей и интеграция с ATS — концепция + базовые меры (лимиты, валидация вилок, жалобы), см. docs/architecture.md.

2. Уточнения заказчика

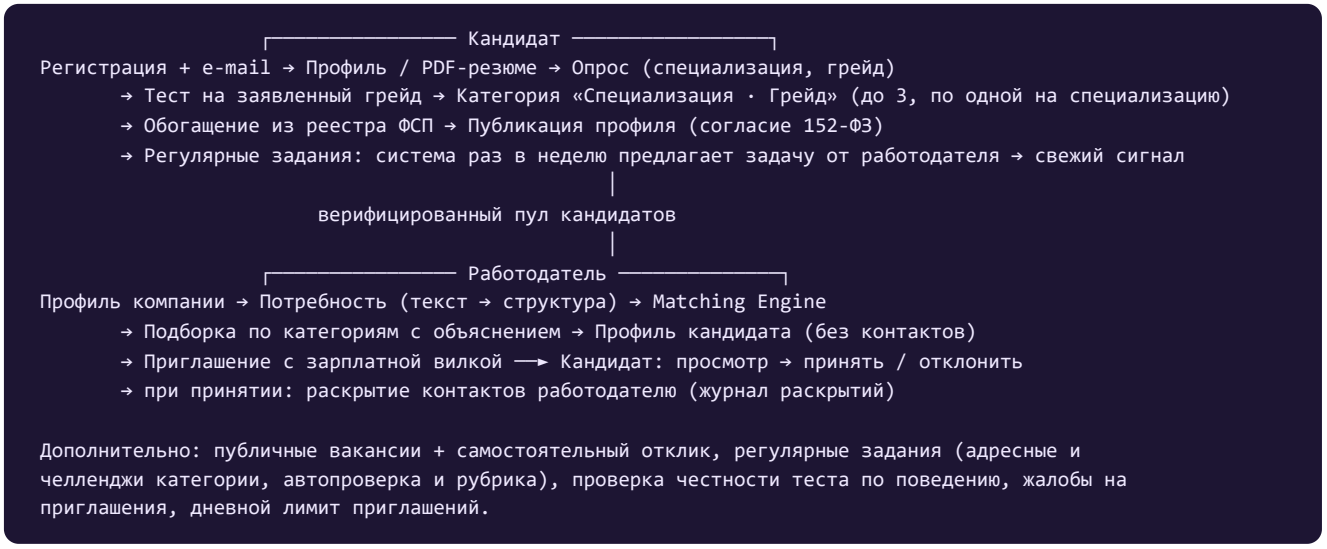
Ответы организаторов трека на вопросы команд и что по каждому изменено в решении. Ссылки ведут на место в документации или код.

#	Вопрос → ответ заказчика (кратко)	Что сделано
1	ФСП даёт буст внутри подтверждённого грейда, но не заменяет тест — **верно**. Дисциплины равноценны; можно учитывать актуальность направления (ИБ-специалист с соревнованиями по ИБ релевантнее, чем с робототехникой)	Логика не менялась: ФСП — 1 из 9 компонентов, грейд подтверждает только тест. Близость дисциплины к специализации потребности уже умножала вклад достижения — теперь она видна работодателю: «профильная / смежная дисциплина / другое направление» у каждого достижения и в объяснении («2 место — … · профильная дисциплина»). Базовые веса от дисциплины не зависят. fsp-integration.md
2	Регулярные задания — **адресно** кандидату; челлендж на категорию не запрещён; способ оценки — на усмотрение команды	Система сама предлагает кандидату задание раз в неделю (одно открытое предложение, срок 7 дней) по правилу: соответствие грейду, работодатель рассматривает кандидата для потребности, к которой привязано задание, задание проверяет слабо подтверждённую компетенцию, новизна, нагрузка на проверку. Кандидат видит, почему задание его. Форматы: автопроверка (число / текст) и подход с предварительной оценкой по ключевым идеям и итоговой рубрикой работодателя (3 критерия × 0–2). Плюс челленджи категории и личное предложение от работодателя. Грейд задания не меняют. testing.md §8
3	Питч — 10 минут; видео вместо стенда допустимо; стенд держать на время оценки	Презентация перестроена под 10 минут (тайминг в заметках к слайдам), демо-видео сценария записывается скриптом `scripts/record_demo.py`; при отсутствии публичного стенда — `docker compose up --build` по deployment.md
4	Процедуру валидации стабильности предлагает команда: повторное прохождение одним профилем (test-retest) со сравнением грейдов + разделяющая сила заданий; симуляция допустима	Добавлено повторное прохождение **всей процедуры** одним профилем на непоказанных заданиях: совпадение грейда 75,9%, тот же или соседний 99,8%, kw 0,866, у уверенных результатов 88,2%. Разделяющая сила: индекс D и r_pb по каждому заданию (47% D ≥ 0,3, 24% — 0,2–0,3), критерии приёмки заданы заранее; скрипт `backend/scripts/item_analysis.py` повторяет анализ на реальных попытках. testing.md §7
5	Грейд — **уровень навыков**, подтверждаемый тестом; стаж — атрибут профиля	Описания грейдов переписаны через навыки (без «опыт до 1–2 лет»); стаж — отдельное поле, в подборке — отдельный фактор «опыт»
6	Одной категории достаточно; несколько — **плюс**	До 3 категорий, по одной на специализацию (`candidate_categories`): тест в новой специализации добавляет категорию, у каждой свои блокировка смены грейда и «быстрая дорожка», основную выбирает кандидат. В подборке кандидата представляет категория специализации потребности; навыки, подтверждённые в другой категории, засчитываются. matching.md §2
7	Облачные LLM допустимы (в т.ч. зарубежные), предпочтительны открытые; риски 152-ФЗ — на команде	LLM в решении не используется: разбор текста — локальные правила (данные не покидают контур). Политика на будущее: открытая модель в своём контуре, в облако — только обезличенный текст; тесты LLM не генерирует. architecture.md §9
8	ФИО и места работы показывать допустимо; ТЗ закрывает только контакты до принятия приглашения / отклика	ФИО видно работодателю по умолчанию (кандидат может скрыть фамилию); контакты — по-прежнему только после согласия, с журналом раскрытий. Добавлены места работы (компания, должность, период, чем занимались): заполняются в профиле или из PDF-резюме, видны работодателю в профиле кандидата и в PDF-профиле

#	Вопрос → ответ заказчика (кратко)	Что сделано
9	Тест — единый под категорию при регистрации, но не одинаковый пул для всех; задания под вакансию — отдельный блок регулярных заданий	Так и устроено: индивидуальный вариант из банка по единому blueprint + параметрические семейства. Задание теперь можно привязать к потребности — оно уходит прежде всего кандидатам из её подборки
10	Лимитов на образ, модели, RAM нет; расход должен быть оправдан	Тяжёлых моделей нет: Python-сервис, PostgreSQL и статический фронтенд на лёгких базовых образах (`python:3.12-slim` , `nginx:alpine`); IRT, подбор и разбор текста считаются на CPU за миллисекунды
11	Качество подбора важнее скорости; «основные операции за приемлемое время»	Замеры: ранжирование 600 кандидатов под потребность — 57 мс, генерация варианта теста — 0,2 мс, оценка попытки — 3 мс (одно ядро)
12	Категории «специализация + грейд» достаточно; детальнее (стек, роль, формат) — плюс	Категория — «специализация · грейд», детализация — в профиле и подборке: стек с отметкой «подтверждён тестом», роли в команде, формат работы, несколько категорий
13	Грейд можно выбрать самому, но без теста — статус «грейд не подтверждён»; официально — только по тесту. Проверка на списывание — большой плюс	Статус заявленного грейда: «подтверждён / подтверждён ниже / **не подтверждён**» (кабинет, профиль; у работодателя в откликах — «Backend · Middle — грейд не подтверждён»). Списывание: к детектору утечки добавлена проверка поведения — уходы со вкладки, вставка, копирование, быстрые верные ответы на трудные для кандидата вопросы; «ищущих ответы на стороне» помечает в 99,8% случаев, «выучивших слитые ответы» — в 84,8%, честных — 0%. testing.md §4a
14	ATS — синхронизация вакансий, откликов, статусов через API; в MVP — концепция (доп. функционал)	Концепция расписана: объекты и направления синхронизации, идемпотентный upsert по внешнему ID, подписанные HMAC вебхуки с повторами, маппинг статусов, исходы найма как метки для дообучения подбора. architecture.md §8
15	Данные — собственные (синтетика / открытые источники); парсинг не как продуктовая механика	Все данные синтетические, вакансии создают работодатели на платформе; парсинга чужих вакансий нет
16	Двух специализаций × двух грейдов достаточно, если сквозной путь работает; широта справочника оценку не поднимает	Справочник не расширялся (4 специализации × 3 грейда); усилия — в механику: категории, задания, проверка честности, валидация
17	Дизайнеров можно включить, приоритет — техническим ролям; идея объективного тестирования дизайнеров — плюс	Концепция: эвристическая оценка макета с эталонным списком дефектов, параметрические расчётные задания (контраст WCAG, сетка, шкалы), дизайн-системы и доступность, UX-метрики; портфолио — атрибут без влияния на грейд. testing.md §9

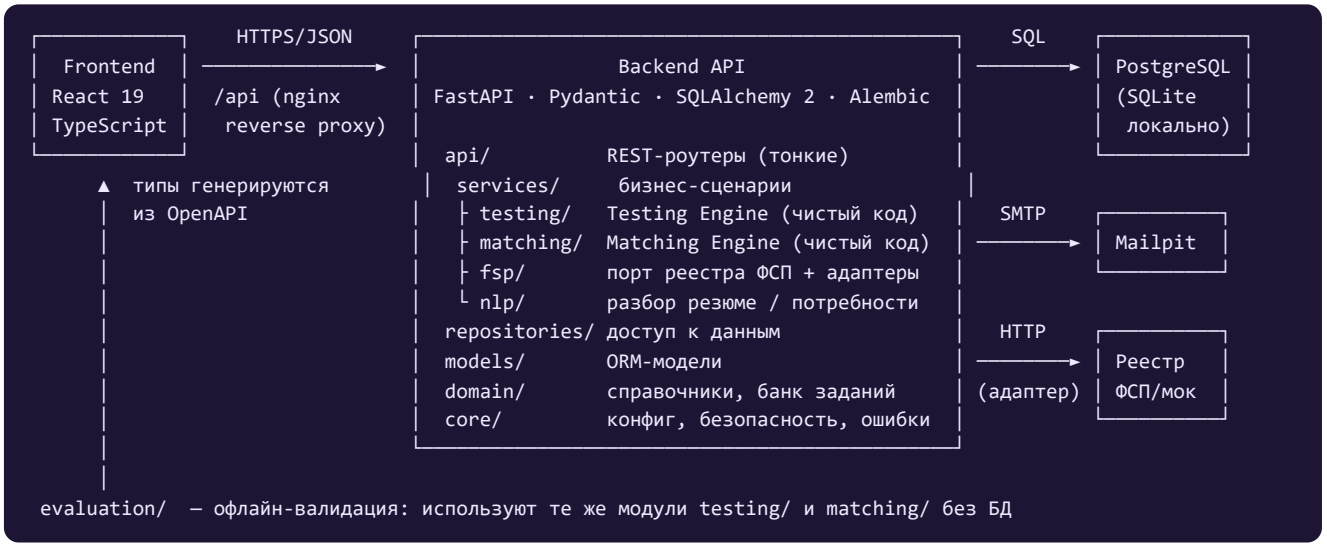
3. Архитектура

1. Функциональная архитектура



Инициатива контакта — у работодателя (модель «обратного подбора», как в LHH): кандидат не рассылает отклики, а получает предложения с условиями.

2. Компонентная архитектура



Монолит с явными слоями и слабо связанными модулями (допускается ТЗ). Testing Engine и Matching Engine — **чистые функции над dataclass-ами** (`ItemSpec` , `CandidateFeatures` , `JobFeatures`), не зависят от БД и HTTP: тот же код работает в API, в seed-данных и в скриптах валидации. Роутеры не содержат бизнес-логики — только валидацию входа (Pydantic) и вызов сервиса.

Слои backend

Слой	Каталог	Ответственность
API	`backend/app/api`	REST-эндпоинты, зависимости аутентификации и ролей, документация ошибок

Слой	Каталог	Ответственность
Схемы	`backend/app/schemas`	Pydantic-контракты запросов/ответов → OpenAPI → TypeScript-типы фронтенда
Сервисы	`backend/app/services`	Сценарии: регистрация, тестирование, подбор, приглашения, приватность
Движки	`services/testing`, `services/matching`	Генерация форм, IRT-оценка, правила грейдов; скоринг, объяснения
Репозитории	`backend/app/repositories`	Запросы с жадной загрузкой для ранжирования целых категорий
Модели	`backend/app/models`	26 таблиц, ограничения целостности (CHECK, UNIQUE, FK)
Домен	`backend/app/domain`	Справочники специализаций/грейдов/навыков, банк заданий

3. Модель данных

```
User 1—1 CandidateProfile 1—* CandidateSkill *—1 Skill
                │ 1—* CandidateCategory (специализация, грейд, θ, компетенции, блокировка; одна основная)
                │ 1—* CandidateExperience (места работы: компания, должность, период «ГГГГ-ММ»)
                │ 1—1 FSPProfile 1—* FSPAchievement
                │ 1—* TestAttempt 1—* TestItem 1—1 TestAnswer        (TestItem → Question)
                │                   1—1 TestResult
User 1—1 EmployerProfile 1—* Job (потребность / вакансия) 1—* JobSkill *—1 Skill
                                  │ 1—* MatchResult *—1 CandidateProfile   (снимок ранжирования)
EmployerProfile 1—* Invitation *—1 CandidateProfile   (Invitation → Job, необязательно)
Job 1—* Application *—1 CandidateProfile
EmployerProfile 1—* ShortTask 1—* TaskAssignment *—1 CandidateProfile   (адресное предложение)
                      │ 1—* TaskSubmission *—1 CandidateProfile    (автопроверка / рубрика)
ShortTask → Job (задание под потребность, необязательно)
ContactAccessLog (employer, candidate, reason) — журнал раскрытий контактов
Specialization 1—* Competency (план теста) · Grade (порог θ) · Question (версионируется)
```

Ключевые решения: * **Job** объединяет «потребность» (приватную, для подбора) и «вакансию» (`is_public`) — приглашать можно без публикации, как требует ТЗ. * **MatchResult** хранит снимок ранжирования: фильтры уточняют подборку, не теряя её и отметки «избранное»; снимок пересчитывается при изменении потребности или по TTL (5 минут). * **TestItem** хранит отрисованный вариант вопроса (перемешанные варианты, параметры семейства) и правильный ответ — он никогда не уходит клиенту. Вопросы банка версионируются (`code` + `version`). * Категории кандидата — `candidate_categories` : по одной на специализацию (Backend · Middle и DevOps · Junior), одна основная; у каждой свои θ, компетенции, блокировка смены грейда и «быстрая дорожка». Поля `verified_*` профиля зеркалят основную категорию. Пишет их **только** сервис тестирования. В подборке кандидата представляет категория специализации потребности (или ближайшая), навыки, подтверждённые в другой категории, тоже засчитываются как доказанные. * Заявленный грейд (опрос) хранится отдельно и показывается со статусом: подтверждён тестом / подтверждён ниже / **не подтверждён** — официально грейд присваивает только тест. * `test_attempts.integrity` — результат проверки честности по поведению, `test_answers.time_ms` и `pasted` — время на вопросе и вставка ответа. * Зарплата обязательна: `CHECK salary_from > 0 AND salary_to >= salary_from` в `jobs` и `invitations` .

Миграции — Alembic (`backend/alembic/versions`), одинаково работают на PostgreSQL и SQLite.

4. Безопасность и приватность

- Пароли — bcrypt; токены — JWT HS256 с OIDC-совместимыми claims (`sub` , `email` , `iss` , `realm_access.roles`). Переход на **Keycloak / ФСП ID** — замена проверки подписи на JWKS (RS256) в `core/security.py::decode_access_token` ; поле `users.external_subject` готово для связи учёток.
- RBAC: зависимости `Candidate` / `Employer` в `api/deps.py` ; владелец ресурса проверяется в сервисах (чужая попытка теста, чужая потребность → 404, чтобы не раскрывать существование).
- Регистрация с подтверждением e-mail (6-значный код, хеш в БД, TTL 30 мин); вход без подтверждения запрещён. В демо-режиме (`EMAIL_DEV_EXPOSE_CODE=true`) код также возвращается API.
- 152-ФЗ: согласие на обработку — обязательно при регистрации; согласие на публикацию — отдельно, без него профиль не участвует в поиске; настройки видимости (фамилия, ФСП, зарплатные ожидания).
- Контакты кандидата раскрываются работодателю **только** после принятия его приглашения или самостоятельного отклика (`services/privacy.py`), каждое раскрытие пишется в `contact_access_log` . ФИО и профиль видны и до этого (по уточнению заказчика закрываются только контакты); кандидат может скрыть фамилию — тогда работодатель видит «Имя Ф.».
- Правильные ответы теста не покидают сервер; ответы валидируются по типу задания.
- Защита от спама: дневной лимит приглашений на работодателя (429), запрет дубликатов активных приглашений (409), ограничение ширины зарплатной вилки (≤ ×3), жалоба кандидата на приглашение.

5. Масштабирование и интеграции

- Stateless API (несколько воркеров uvicorn), состояние — в PostgreSQL. Ранжирование категории — O(N) по верифицированным кандидатам смежных специализаций с жадной загрузкой навыков/ФСП; снимок кэшируется в `match_results` . Для десятков тысяч кандидатов — предварительный отбор по категории в SQL и фоновой пересчёт снимков.
- Интеграция с реестром ФСП — через порт `FSPRegistry` (см. fsp-integration.md).
- **ATS (концепция).** См. раздел 8.
- **Защита от фиктивных вакансий (концепция).** Скоринг доверия работодателя: верификация домена e-mail/ИНН, доля принятых приглашений, жалобы кандидатов (`invitations.reported`), аномалии вилок относительно рынка категории; при низком доверии — модерация приглашений и снижение лимита. В MVP реализованы: лимиты, валидация вилок, жалобы.

6. Используемые библиотеки (версии)

Backend (Python 3.12): fastapi 0.142.2, uvicorn 0.54.0, sqlalchemy 2.1.3, alembic 1.20.0, pydantic 2.13.5, pydantic-settings 2.15.0, psycopg 3.3.6, pyjwt 2.15.1, bcrypt 5.0.0, email-validator 2.3.0, python-multipart 0.0.32, pypdf 6.19.0, httpx 0.28.1; dev: pytest 9.1.1, ruff 0.16.10.

Frontend (Node 22): react 19.3.0, react-dom 19.3.0, react-router-dom 7.18.4, lucide-react 1.52.0, vite 8.3.2, typescript 5.9.3, @vitejs/plugin-react 6.1.1, openapi-typescript 7.13.0, eslint 10.12.0, typescript-eslint 8.71.0; тесты: vitest 5.0.3, @vitest/coverage-v8 5.0.3, jsdom 30.1.2, @testing-library/react 16.3.3, @testing-library/dom 10.4.2, @testing-library/jest-dom 7.0.1, @testing-library/user-event 14.6.7.

Инфраструктура: PostgreSQL 16 (alpine), nginx 1.27 (alpine), Mailpit 1.21, Docker Compose. Все компоненты — open source (MIT / Apache-2.0 / BSD / PostgreSQL License).

7. Локализация (RU / EN)

Выбор языка. Порядок: явный выбор на устройстве (переключатель RU / EN, `localStorage`) → язык учётной записи (`users.locale`, восстанавливается при входе с другого устройства) → язык браузера (`navigator.languages`) → русский. Выбор, сделанный вошедшим пользователем, сохраняется в учётной записи (`PUT /api/auth/me/locale`); при регистрации записывается язык, на котором она прошла.

Фронтенд (`frontend/src/i18n`). Словари по разделам (`locales/ru/*`, `locales/en/*`); английский словарь типизирован от русского, поэтому пропущенный ключ — ошибка компиляции. Плюрализация — `Intl.PluralRules` (ru: one / few / many, en: one / other), числа, деньги и даты — `Intl` (`ru-RU`, `en-GB`: порядок «день-месяц-год» в обоих языках). API-клиент отправляет `Accept-Language` с каждым запросом, а `useApi` перезапрашивает данные при смене языка — тексты, сформированные сервером (объяснения, причины, справочники), переключаются вместе с интерфейсом. `<html lang>` обновляется для скринридеров. Тест `coverage.test.ts` запрещает русский текст в компонентах вне словарей.

Бэкенд (`backend/app/i18n`). ASGI-middleware определяет язык запроса по `Accept-Language` (с учётом q-весов) и кладёт его в контекстную переменную; сервисы вызывают `t(key, **params)`. Ответ помечается `Content-Language` и `Vary: Accept-Language`. Ошибки 422 от Pydantic переводятся с названием поля.

Хранимые тексты не хранятся готовыми. Снимок подборки (`match_results`) и причины в приглашении сохраняются как объекты сообщений `{code, params}` с языконезависимыми параметрами (id, числа, имена собственные; деньги — `{"money": n}`, навык — `{"skill": slug}`, дата — `{"date": …}`) и отрисовываются при чтении: работодатель на английском и кандидат на русском видят одну и ту же подборку каждый на своём языке. Рекомендация к результату теста выводится из сохранённого исхода. Старые записи с готовым текстом отображаются как есть.

Банк заданий. Все 252 статических задания и 25 параметрических семейств есть на английском. Перевод хранится в `questions.i18n` с тем же порядком вариантов, поэтому ключ ответа и IRT-калибровка общие; семейства генерируют обе версии из одних случайных параметров. В попытке (`test_items.i18n`) хранятся обе версии с одинаковой перестановкой вариантов — язык можно сменить посреди теста, ответы остаются верными. Тесты проверяют полноту перевода, совпадение числа вариантов, неизменность фрагментов кода и одинаковые числа в обеих версиях параметрических заданий.

Разбор текста. NLP-разбор потребности и резюме понимает оба языка: «желательно Redis» и «Redis is a plus», «190–260 тыс.» и «from 150 to 200k», «опыт от 2 лет» и «2+ years».

Пользовательские данные (резюме, описания вакансий, названия соревнований из реестра ФСП) показываются так, как их ввёл автор.

8. Интеграция с ATS работодателя (концепция)

По уточнению заказчика ATS-интеграция — это синхронизация вакансий, откликов и статусов с системой найма работодателя через API, чтобы платформа встраивалась в существующий процесс. В MVP не требуется; ниже — архитектурный подход, на который уже рассчитана модель данных.

Объект	Направление	Как
Потребность / вакансия	ATS → платформа	`PUT /integrations/v1/requirements/{external_id}` — идемпотентный upsert по внешнему ID; текст разбирается тем же NLP, что и форма
Подборка	платформа → ATS	`GET /integrations/v1/requirements/{external_id}/matches` — ранжирование с объяснениями (контакты закрыты)

Объект	Направление	Как
Приглашение / отклик	платформа → ATS	вебхуки `invitation.accepted`, `application.created` — профиль кандидата в стандартизированном JSON (как PDF-профиль) с контактами: на этом шаге кандидат дал согласие
Статусы	ATS → платформа	`POST /integrations/v1/applications/{id}/status` (interview / offer / hired / rejected) — кандидат видит статус; исходы — метки для дообучения весов подбора
Задания	ATS → платформа	создание регулярного задания под потребность тем же контрактом, что `POST /employer/tasks`

- Аутентификация — API-ключ компании (хранится хешем, отзывается в кабинете) или OAuth2 client credentials; права — только своя компания.
- Вебхуки подписываются HMAC-SHA256 (`X-FSP-Signature`), доставка at-least-once с повторами и экспоненциальной задержкой, получатель дедуплицирует по `event_id`; журнал доставок в кабинете.
- Маппинг статусов настраивается (у каждой ATS свой справочник этапов); неизвестный статус не ломает синхронизацию, а сохраняется как «другое».
- Приватность не меняется: контакты уходят в ATS только после согласия кандидата (принятое приглашение или отклик), каждая передача пишется в `contact_access_log`.

9. NLP и LLM

Разбор резюме и описания потребности (RU / EN) — правила и словари (`services/nlp`), работают локально, персональные данные не покидают контур. Заказчик допускает облачные LLM (в т.ч. зарубежные, предпочтительнее открытые модели); если подключать модель для разбора свободного текста, то открытую (класса 7–14B параметров) в своём контуре, а в облачную — только обезличенный текст (ФИО, контакты и названия компаний заменяются метками до вызова), чтобы не возникала трансграничная передача персональных данных по 152-ФЗ. Тесты и их оценка LLM не используют: генерация заданий нейросетью не гарантирует сопоставимую сложность (ТЗ), поэтому задания калиброваны по IRT.

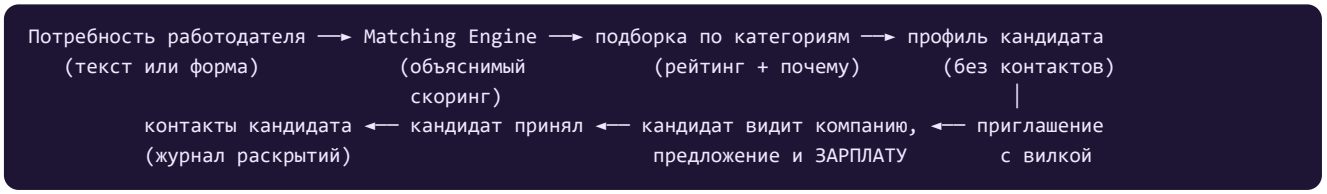
10. Фоновые задачи

Регулярные задания предлагаются фоновым проходом раз в `TASK_OFFER_LOOP_MINUTES` (по умолчанию 60) и дополнительно при открытии кандидатом своих заданий или кабинета; проход идемпотентен (одно открытое предложение на кандидата, уникальный индекс «задание–кандидат»), поэтому несколько воркеров не создают дублей. Для продакшна с несколькими инстансами — вынести в планировщик (cron / очередь).

4. Механика подбора

1. Сценарий: обратный подбор

Кандидат **не откликается** на вакансии. Инициатива — у работодателя:



- Потребность не обязана быть опубликованной вакансией: приглашать можно напрямую.
- Описание потребности можно ввести текстом — NLP-разбор извлекает специализацию, грейд, стек, вилку, формат и опыт (`services/nlp/extractor.py`); работодатель проверяет и уточняет поля. Навыки после «желательно / приветствуется» или перед «будет плюсом» попадают в nice-to-have, остальные — в must-have; вилка «190–260 тыс.» понимается целиком, телефон не принимается за зарплату.
- Подборка сохраняется (`match_results`), фильтры уточняют её **без потери** (грейд, категория, обязательный навык, наличие ФСП, минимальный %, избранное); пересчёт — при изменении потребности, по TTL или кнопкой.
- Работодатель видит «рекомендованные категории» (Backend · Middle — 10 кандидатов, средний и максимальный %), затем конкретных людей в категориях.
- Сравнение «почему A выше B» — разложение разницы по факторам (`GET …/compare`).
- Приглашение: должность, описание, **обязательная вилка от–до в ₽**, формат, способ связи; статусы `sent → viewed → accepted | rejected`, а также `expired` (14 дней) и `withdrawn`.
- Кандидат видит условия и «почему выбрали вас» до ответа; контакты раскрываются только после принятия (или если кандидат сам откликнулся на вакансию).

Дополнительно: банк кандидатов (вся база с фильтрами, сортировка внутри категории по силе подтверждённого профиля), публичные вакансии с обратным матчингом для кандидата (% совпадения у каждой вакансии), отклики, короткие задания.

2. Категоризация

Категория = **подтверждённая специализация · подтверждённый грейд** (Backend · Middle и т.д.), присваивается только тестированием (см. testing.md). Кандидат может подтвердить до трёх категорий — по одной на специализацию (Backend · Middle и DevOps · Junior). В подборке кандидата представляет категория специализации потребности (если её нет — ближайшая по матрице смежности специализаций): её грейд, θ и компетенции идут в формулу, а навыки, подтверждённые тестом в другой категории, тоже считаются доказанными. Кандидат без подтверждённой категории или без согласия на публикацию в подборки не попадает. Внутри категории кандидаты ранжируются по силе подтверждённого профиля — ФСП усиливает ранжирование внутри подтверждённого грейда, но не заменяет тест (заказчик подтвердил эту логику):

```
profile_strength = 0.6 · σ(1.7·(θ − порог грейда)) + 0.25 · ФСП + 0.15 · актуальность
```

3. Формула совпадения

```
match = gate_must × Σ_k w_k · s_k / Σ_k w_k        по ДОСТУПНЫМ компонентам k,  s_k ∈ [0, 1]
```

k	Компонент	Вес	Как считается
skills	Стек и навыки	0.26	0.75·покрытие must + 0.25·nice. Каждый требуемый навык ищется через **граф навыков** (FastAPI ≈ Flask 0.7, PostgreSQL ≈ MySQL 0.65, React ≈ Next.js 0.75 …) и умножается на **доказательность**: заявленный уровень × (0.75 + 0.25·балл теста по компетенции, которая этот навык проверяет)
grade	Грейд	0.22	совпадает 1.0; выше на 1 — 0.6; ниже на 1 — 0.4 (0.6 если тест показал готовность к повышению); разница ≥ 2 — исключение
test	Результат теста	0.16	0.65·σ(1.7·(min(θ, порог+1) − порог требуемого грейда)) + 0.35·средний балл компетенций, релевантных стеку. Способность выше следующего грейда не даёт бонуса (переквалификацию учитывает «грейд»)
specialization	Специализация	0.10	совпадает 1.0; смежная — частично (Backend↔DevOps 0.35, Backend↔Data 0.25…); несмежная — исключение
fsp	Достижения ФСП	0.10	0.3 + 0.7·сила; **нет истории → 0.3 (нейтрально, без штрафа)**
freshness	Актуальность профиля	0.06	0.5^(дней с последней активности / 120); активность — тест, ответ на приглашение, короткое задание
salary	Зарплатные ожидания	0.05	в вилке — 1; выше — линейный спад до 0 при превышении на 40%; не указаны — компонент недоступен
experience	Опыт	0.03	min(1, опыт / требуемый); требование не задано — недоступен
format	Формат работы	0.02	матрица совместимости remote/office/hybrid + город

- **Нет данных ≠ 0.** Недоступные компоненты (не указаны ожидания по зарплате, нет требования к опыту) исключаются, а веса перенормируются. ФСП всегда «доступен» через нейтральную базу.
- **Мягкое «И» для обязательных навыков**: `gate_must = min(1, (0.8 + 0.2·покрытие_must) / 0.92)` — отсутствие обязательных навыков масштабирует весь балл, чтобы сильный тест не маскировал отсутствие того, что работодатель явно требует.
- Жёсткие фильтры: несмежная специализация, разница грейдов ≥ 2, итог ниже 35%.
- При равенстве баллов выше тот, у кого выше θ, затем — сильнее ФСП.

Профили весов на выбор работодателя: **Сбалансированный** (по умолчанию), **Сначала стек** (skills 0.40), **Подтверждённый талант** (test 0.30, ФСП 0.18). Веса переопределяются через `MATCHING_WEIGHTS_JSON`. Код: `backend/app/services/matching/engine.py`, `config.py`.

Почему такие веса

- Стек (0.26) — главный язык, которым работодатель описывает потребность, но навык учитывается по доказательности, а не по факту упоминания в резюме.
- Грейд (0.22) — центральная часть категории; подобран на бенчмарке (п. 6): недо- и переквалификация — основные причины нерелевантной выдачи.
- Тест (0.16) — сопоставимая (IRT) мера уровня относительно требуемого грейда.
- Специализация (0.10) — в основном работает как фильтр, частичный балл только для смежных профилей.
- ФСП (0.10) — проверяемый сигнал практических навыков; заметно поднимает внутри категории, но не может перевесить стек и грейд; нейтральная база исключает дискриминацию тех, кто не участвовал.

- Актуальность, зарплата, опыт, формат — небольшие поправки (самоописанный опыт — сознательно 0.03).

4. Объяснимость

Каждый результат содержит: * **причины** — человекочитаемые утверждения с типом (✓ плюс / • нейтрально / ✗ минус): «Подтверждённый грейд Middle — как требуется», «Совместимость стека 82%: 4 из 4 обязательных навыков, 4 подтверждены тестом», «Близкие навыки: Flask вместо FastAPI», «Нет истории ФСП — сигнал не учитывается, без штрафа», «Ожидания 410 000 ₽ — выше вилки на 64%»; * **разложение балла** — оценка, вес и вклад каждого фактора в итоговые проценты (сумма вкладов = итог), множитель покрытия обязательных навыков; * **сопоставление навыков** — точное совпадение / близкий навык / отсутствует, подтверждён ли тестом; * **сравнение двух кандидатов** — разница вкладов по факторам и текстовое резюме («Разница +8 п. Преимущество: грейд (+8.8), стек и навыки (+2.3). Уступает: достижения ФСП (−1.2)»).

- **свежий сигнал** — если кандидат недавно выполнил регулярное задание: «Свежий сигнал: задание „Идемпотентность платежей“ — 83%, 3 дня назад»; на карточке — «Задания: 3 · средний балл 83%»;
- **достижения ФСП с профильностью** — «2 место — Всероссийские соревнования… (2026) · профильная дисциплина»: все дисциплины ФСП равноценны, но вес достижения зависит от близости дисциплины к специализации потребности (см. fsp-integration.md).

Кандидат в приглашении видит процент совпадения и положительные причины — почему выбрали его.

5. Приватность в подборе

До контакта работодатель видит ФИО (кандидат может скрыть фамилию — «Имя Ф.»), категории, тест, навыки, ФСП, город, ожидания (если кандидат разрешил). Контакты — только после принятия приглашения или отклика. Скрытые кандидатом данные (ФСП, зарплата) не используются и в ранжировании.

6. Валидация (`python evaluation/evaluate_matching.py`)

Готового датасета нет, поэтому построен **синтетический бенчмарк со скрытой истиной**: 60 потребностей × 600 кандидатов. У кандидата есть скрытый «истинный» профиль (специализация, способность θ, реальные навыки); движок видит только наблюдаемое:

- резюме: истинные навыки с пропусками (10%) и **«приписанными» модными навыками своей области** у 40% кандидатов (keyword stuffing — именно проблема, описанная в ТЗ);
- самооценка грейда: завышение на уровень у 30%, занижение у 10%;
- результат теста: θ с погрешностью 20-заданной формы (SE ≈ 0.33) и баллы компетенций;
- ФСП: у ~50%, сильнее у сильных; зарплатные ожидания, формат, активность.

Метки релевантности 0–3 назначаются **экспертными правилами по скрытому профилю** (полезность найма: специализация × соответствие истинного грейда × реальное покрытие must-have × сила внутри грейда × совместимость по зарплате) — без использования весов движка. Релевантен кандидат с меткой ≥ 2.

Ранжировщик	P@5	P@10	R@10	MRR	NDCG@10
Случайный (та же специализация)	0.193	0.202	0.062	0.311	0.168
Поиск по ключевым словам резюме («классическая площадка»)	0.377	0.412	0.169	0.655	0.461

Ранжировщик	P@5	P@10	R@10	MRR	NDCG@10
Ключевые слова + заявленный грейд	0.700	0.608	0.273	0.817	0.692
FSP Talent (сбалансированный)	**0.880**	**0.807**	**0.346**	**0.933**	**0.808**
— без тестирования (самооценка грейда)	0.813	0.747	0.325	0.929	0.771
— только подтверждённый грейд (без θ и компетенций)	0.887	0.817	0.352	0.992	0.833
— без графа навыков	0.873	0.782	0.343	0.958	0.824
— без сигнала ФСП	0.890	0.818	0.352	0.942	0.819
— без мягкого «И» для must-have	0.883	0.808	0.345	0.933	0.804
Профиль «Сначала стек»	0.717	0.662	0.302	0.795	0.674
Профиль «Подтверждённый талант»	0.673	0.625	0.287	0.816	0.631

Выводы: * Движок существенно превосходит поиск по ключевым словам (NDCG@10 0.81 против 0.46) и даже поиск с учётом самооценки грейда (0.69): в топ-5 релевантны 88% кандидатов против 38% и 70%. * Тестирование даёт прирост: с самооценкой грейда вместо подтверждённого NDCG падает до 0.77. * Честно о слабых местах: в этой разметке сила внутри грейда влияет слабо (±15%), поэтому упорядочивание по θ внутри категории даёт −0.025 NDCG относительно «только грейд»; граф навыков и ФСП в синтетике почти нейтральны (метки не содержат ФСП, «близкие» навыки редки). Эти компоненты оставлены ради объяснимости и требований ТЗ («выше те, у кого сильнее подтверждённый профиль и есть опыт ФСП»), их веса умеренные. * **Справедливость к кандидатам без ФСП**: идеальные (метка 3) кандидаты без истории ФСП в среднем на 4,5-м перцентиле выдачи, с ФСП — на 3,6-м: ФСП помогает, но отсутствие не «топит».

Подбор гиперпараметров (`python evaluation/tune_matching.py`)

Сетка (минимум мягкого «И», сила тестовой доказательности навыков, доля компетенций в компоненте теста, 3 варианта весов) на обучающем мире (seed 11), проверка на **независимом** мире (seed 23):

Конфигурация	NDCG@10 train	NDCG@10 held-out	P@10 held-out	MRR held-out
Исходная экспертная (v1.0)	0.624	0.594	0.573	0.797
Лучшая на сетке (gate 0.8, evidence 0, grade 0.22)	0.832	0.809	0.792	0.967
Принятая в продукт (v1.2, evidence 0.25)	**0.808**	**0.786**	**0.770**	**0.956**

Прирост на независимых данных сохраняется (0.594 → 0.786), значит подбор не переобучен на одну синтетическую выборку. Сила тестовой доказательности навыков оставлена 0.25 вместо оптимальных 0: «навыки, подтверждённые тестом, весят больше» — продуктовый стимул проходить тест; цена — 0.023 NDCG на отложенных данных. Полный отчёт: `evaluation/results/tuning_report.md`.

Скорость: ранжирование всего пула (600 кандидатов) под одну потребность — признаки, 9 компонентов и объяснения — **57 мс** на одном ядре; подборка сохраняется снимком (заказчику важнее качество подбора, чем скорость; основные операции укладываются в доли секунды).

Свой набор пар «вакансия — кандидат» можно проверить командой `python evaluation/evaluate_matching.py --dataset my_pairs.json` (формат — как у `evaluation/data/matching_benchmark.json`, экспортируемого скриптом).

7. Ограничения и развитие

- Веса подобраны на синтетике; после запуска — дообучение на реальных исходах (приглашение → принятие → найм) как learning-to-rank с теми же признаками; объяснимость сохраняется (линейная модель над интерпретируемыми компонентами).
- Граф навыков — экспертный; может пополняться эмбеддингами совместной встречаемости навыков.
- Регулярные задания сейчас влияют на актуальность и дают «свежий сигнал» в объяснении; когда накопится статистика, оценки можно сделать отдельным компонентом «подтверждённые практические задачи» с весом, подобранным на исходах приглашений.

5. Механика тестирования

Требования ТЗ и как они закрыты

Требование	Решение
Единый набор заданий не подходит — разойдётся	Каждому кандидату генерируется свой вариант из банка (сейчас 252 статических задания + 25 параметрических семейств)
Генерация нейросетью не гарантирует сопоставимую сложность	Варианты собираются по **единому плану (blueprint)**: одинаковое число заданий по компетенциям и одинаковый профиль сложности; оценка по **IRT** учитывает сложность конкретных заданий
Категория — не из самоописания, а через опрос → выбор грейда → тест	Категория `Специализация · Грейд` меняется только сервисом тестирования
Грейд не понижается принудительно; можно тест ниже/выше	Провал заявленного грейда → тест ниже доступен сразу; уверенный результат → тест выше без ожидания
Смена грейда ограничена по времени	Не чаще раза в 90 дней (`grade_locked_until`), повтор того же теста — через 30 дней
Грейд — уровень навыков, подтверждённый тестом (уточнение заказчика)	Заявленный в опросе грейд показывается со статусом «подтверждён / подтверждён ниже / **не подтверждён**»; официально грейд присваивает только тест, стаж — отдельный атрибут профиля
Несколько категорий — плюс (уточнение)	До 3 категорий, по одной на специализацию; у каждой своя блокировка и «быстрая дорожка», одна основная
Проверка на списывание — большой плюс (уточнение)	Детектор утечки (что ответил) + проверка поведения (как отвечал): уходы со вкладки, вставка, копирование, быстрые верные ответы на трудные вопросы
Процедура валидации — тест-ретест и разделяющая сила заданий (уточнение)	Раздел 7: повторное прохождение всей процедуры одним профилем, индекс D и r_pb по каждому заданию, скрипт для реальных данных
Регулярные задания — адресно, оценка на усмотрение команды (уточнение)	Раздел 8: система предлагает задачу конкретному кандидату; автопроверка или рубрика работодателя

1. Банк заданий

`backend/app/domain/question_bank/` — 4 специализации × 5–6 компетенций, уровни сложности 1–5:

Специализация	Компетенции (доля в плане теста)
Backend	язык и рантайм 22%, БД и SQL 20%, API/HTTP/безопасность 18%, архитектура 18%, алгоритмы 12%, эксплуатация 10%
Frontend	JS/TS 26%, React и состояние 24%, HTML/CSS/a11y 18%, браузер и производительность 18%, тесты и сборка 14%
Data / ML	классическое ML 26%, статистика 20%, данные (SQL, pandas) 20%, DL 18%, MLOps 16%
DevOps	контейнеры и оркестрация 24%, Linux и сети 22%, CI/CD и IaC 20%, мониторинг 18%, облака и безопасность 16%

Метаданные задания (`questions`): `code` , `version` , `specialization` , `competency` , `difficulty` , `qtype` (single / multi / numeric), варианты, правильный ответ, IRT-параметры `a` , `b` , `c` , счётчик показов `exposure_count` , `family` для параметрических шаблонов. Изменение содержимого создаёт новую версию (старая деактивируется, попытки ссылаются на точную версию).

Параметрические семейства (`families.py` , `families_extra.py`) — шаблоны с фиксированной когнитивной операцией и сложностью, у которых при каждом показе меняются параметры: длина `range()` , число сравнений бинарного поиска, асимптотика циклов, строки после `GROUP BY … HAVING` , `LEFT JOIN` , специфичность CSS-селектора, батчинг `setState` , semver-диапазоны, precision/recall/F1, теорема Байеса, размер выхода свёртки, CIDR, chmod, бюджет простоя SLO, ёмкость узла Kubernetes, формула HPA и др. Ответ одного кандидата бесполезен следующему: у него другие числа. Семейства покрывают все 21 компетенцию.

Языки. Каждое задание банка есть на русском и английском (`question_bank/en_*.py` , по коду задания). Варианты ответа идут в том же порядке, фрагменты кода не меняются, поэтому правильный ответ и параметры IRT общие. Параметрические семейства формируют обе версии из одного набора случайных параметров. Кандидат проходит тест на языке интерфейса и может переключить язык посреди попытки — вопросы придут в переводе, данные ответы останутся засчитанными.

2. Генерация варианта (`services/testing/generator.py`)

1. **Blueprint** для (специализация, целевой грейд, длина = 20): * квоты компетенций — по долям плана (метод наибольших остатков, минимум 1); * профиль сложности сконцентрирован у порогов решения грейда (там нужна максимальная информативность теста):

Цель	ур.1	ур.2	ур.3	ур.4	ур.5
Junior	20%	40%	33%	7%	—
Middle	—	20%	40%	33%	7%
Senior	—	—	25%	45%	30%
* уровни распределяются по компетенциям детерминированно (Сент-Лагю), так что у каждой					
компетенции есть лёгкие и трудные слоты. **План одинаков для всех кандидатов.**					
2. **Заполнение слотов**: для каждого слота случайно выбирается задание той же компетенции и					
сложности; приоритет — редко показанным (вес `1/(1+показы)` , контроль экспозиции), параметрические					
семейства предпочитаются (вес 3, утечка им не страшна). Задания, уже виденные кандидатом, исключаются.					
Если ячейка пуста — берётся ближайший уровень в сторону, компенсирующую накопленный сдвиг					
сложности, поэтому средняя сложность формы остаётся равной плановой.					
3. **Отрисовка**: случайный порядок заданий, перемешанные варианты ответов, новые параметры семейств.					
Правильные ответы хранятся на сервере (`test_items.correct`) и никогда не отправляются клиенту.					

3. Оценка (`services/testing/irt.py` , `scoring.py`)

- Модель 3PL: `P(верно | θ) = c + (1 − c) / (1 + e^(−1.7·a·(θ − b)))` ; начальные параметры — из экспертного уровня сложности (`b ∈ {−2, −1, 0, 1, 2}`) и формата (`a` = 1.0 / 1.3 / 1.4, `c` = 0.8/k для выбора из k, 0.05 для множественного выбора, 0 для числового ответа).

- Способность θ оценивается методом **EAP** (апостериорное среднее на сетке $\theta \in [-4; 4]$, априорно $N(0; 1.25^2)$) — конечна даже при всех верных/неверных ответах; апостериорное СКО — погрешность.
- Пороги грейдов на шкале θ: Junior −1, Middle 0, Senior +1. Публичная шкала **0–100 = 50 + 25·θ**, пороги читаются как 25 / 50 / 75 баллов. Шкальный балл сопоставим между разными вариантами, в отличие от «% верных».
- **Решение**: грейд подтверждён, если `P(θ ≥ порог | ответы) ≥ 0.5`; «уверенный результат» (открывает тест на грейд выше без ожидания), если `P(θ ≥ порог следующего грейда) ≥ 0.8`. Уверенность решения — соответствующая апостериорная вероятность.
- Профиль компетенций — доля баллов по каждой компетенции (с частичным зачётом множественного выбора).
- Категория = `верифицированная специализация · подтверждённый грейд`.

4. Детектор утечки (защита от «слитых» ответов)

Кандидат, выучивший ответы, отвечает неправдоподобно: верно на трудные «известные» задания и неверно на свежие. Два статистических признака (`scoring.py`):

- **person-fit lz*** (Drasgow, 1985) — стандартизованное правдоподобие паттерна ответов; порог −2;
- **разрыв способностей**: θ по статическим заданиям минус θ по параметрическим (неутекаемым), в единицах погрешности; порог 1.9.

Помеченный результат не присваивает грейд: кандидату сразу доступна **одна подтверждающая форма** из непоказанных ему заданий (без кулдауна). Честные кандидаты помечаются в 4,7% случаев и просто проходят ещё одну форму.

4a. Проверка честности по поведению (`services/testing/integrity.py`)

Детектор утечки смотрит, **что** ответил кандидат; эта проверка — **как** он отвечал. Страница теста открыто (кандидат предупреждён в начале теста; камера, микрофон и экран не используются) передаёт с каждым ответом:

- уходы со вкладки теста — число и суммарное время вне вкладки;
- копирование текста вопроса и вставку ответа из буфера обмена;
- время на вопросе, пока вкладка видима.

При завершении сервер складывает баллы риска: уходы со вкладки (3+ → 1, 8+ → 2), доля времени вне вкладки ≥ 25% → 1, вставки (1+ → 1, 3+ → 2), копирования 2+ → 1, и **быстрые верные ответы на вопросы, трудные для этого кандидата**: P(верно | θ_ref) < 0,35, а ответ дан быстрее четверти ожидаемого времени задания (2+ → 1, 4+ → 3). Это сигнатура заранее известного ответа по времени реакции (van der Linden & Guo, 2008). θ_ref оценивается **только по параметрическим заданиям** — их ответы нельзя выучить, поэтому утечка не завышает эталонную способность.

Итог: `ok` (0 баллов), `attention` (1–2: замечание без последствий) или `risk` (3+). Один сигнал ничего не решает. Результат с риском не отклоняется: как и при флаге утечки, кандидату сразу доступна одна подтверждающая форма, грейд не теряется. Кандидат видит, что именно замечено. Счётчики накопительные, сервер хранит максимум — перезагрузка страницы их не сбрасывает.

5. Правила грейдов и кулдауны (`services/testing/policy.py`)

- одна активная попытка; лимит времени 30 минут (просроченные завершаются автоматически с данными ответами);
- тот же (специализация, грейд) — повтор через 30 дней;
- без подтверждённого грейда: провалили Middle → тест на Junior доступен сразу (профиль не блокируется и не удаляется);
- подтверждённый грейд меняется не чаще раза в 90 дней, кроме «быстрой дорожки»: уверенный результат открывает тест на следующий грейд сразу;
- проваленный тест на повышение **не понижает** грейд; переподтверждение текущего грейда обновляет актуальность профиля;
- тест в другой специализации **добавляет** категорию (до 3), не заменяя имеющуюся; у каждой категории свои блокировка смены грейда и «быстрая дорожка»; основную кандидат выбирает сам.

Все сроки настраиваются переменными окружения.

6. Устойчивость к распространению заданий — итог

1. Вариант у каждого свой (пересечение соседних вариантов ~30–50% по статическим заданиям), порядок и варианты ответов перемешаны.
2. Параметрические семейства не утекают по построению; их доля в варианте повышена.
3. Контроль экспозиции распределяет показы по банку; исключаются уже виденные задания.
4. IRT позволяет постоянно пополнять банк: новые задания калибруются по ответам (см. п. 7 валидации) и сразу становятся взаимозаменяемыми со старыми.
5. Детектор утечки + подтверждающая форма.
6. Проверка поведения: уходы со вкладки, вставка, быстрые верные ответы на трудные вопросы (п. 4а).
7. Регулярные задания от работодателей дают независимый «свежий» сигнал (п. 8).

7. Валидация (`python evaluation/evaluate_tests.py`)

Процедура

Эталонной разметки нет, поэтому валидность проверяется **симуляцией с известной истиной** — это допускает и заказчик («допустима симуляция на синтетических кандидатах»):

1. **Популяция.** 400 синтетических кандидатов на каждую из 4 специализаций с истинной способностью θ и, значит, истинным грейдом ($\theta < -1$ — ниже Junior, $[-1; 0)$ — Junior, $[0; 1)$ — Middle, ≥ 1 — Senior); 10% ниже Junior, 20% заявляют грейд выше реального.
2. **Ответы без круговой логики.** Ответы генерируются по 3PL с **истинными** параметрами заданий, которые отличаются от номинальных (`b + N(0; 0,35)` , `a · LogN(0; 0,2)`); движок оценивает по номинальным — проверка включает ошибку модели.
3. **Продакшн-код целиком:** blueprint → генерация варианта → оценка → правила грейдов → подтверждающая форма при флаге.
4. **Метрики** по критериям ТЗ: соответствие эталону (истинному грейду), стабильность грейда при повторном прохождении, разделяющая сила заданий, устойчивость к утечке и списыванию, эквивалентность вариантов.
5. **Критерии приёмки**, заданные до прогона: взвешенная κ с эталоном $\geq 0{,}8$; завышение грейда $\leq 5\%$; повторное прохождение — тот же грейд или соседний $\geq 95\%$; $\geq 2/3$ заданий с $D \geq 0{,}2$; пройденность «слитого» теста с детектором $\leq 25\%$ при одном слитом варианте; ложные пометки честных $\leq 5\%$.

6. **На реальных данных** та же процедура повторяется на пилоте: `python backend/scripts/item_analysis.py` считает p, r_pb и индекс D по каждому заданию из сохранённых попыток и помечает задания на пересмотр; тест-ретест — добровольцы проходят тест повторно через 2–4 недели; экспертная валидация — сравнение грейда с оценкой технического интервью на выборке нанятых.

Результаты (seed = 7, длина теста 20)

Метрика	Значение	Критерий
Точность итогового грейда (= истинному)	**82,6%**	
Взвешенная к Коэна с эталоном	**0,904**	≥ 0,8 ✓
Грейд в пределах ±1	99,9%	
Завышение грейда (риск для работодателя)	2,9%	≤ 5% ✓
Занижение (кандидат может пересдать / пройти ниже)	14,4%	
Эквивалентность форм: SD ожидаемого балла у порога, blueprint / случайный отбор	**0,013 / 0,045**	
AUC Middle vs Junior · Senior vs Junior	0,943 · 0,999	
Средний шкальный балл: ниже Junior / Junior / Middle / Senior	14 / 37 / 62 / 87	

Стабильность грейда при повторном прохождении (тест-ретест)

Тот же профиль **дважды проходит всю процедуру** (тест на заявленный грейд → при неудаче тест ниже → подтверждающая форма при флаге), во второй раз — на заданиях, которых ещё не видел. Сравниваются присвоенные грейды:

Метрика	Значение
Совпадение присвоенного грейда	**75,9%**
Тот же или соседний грейд	**99,8%** (критерий ≥ 95% ✓)
Взвешенная к между прохождениями	**0,866**
Совпадение, если оба результата уверенные (≥ 0,8; таких 57%)	**88,2%**
Совпадение, если θ дальше 0,25 от порога грейда (59% кандидатов)	82,6%
Совпадение у пограничных (|θ − порог| < 0,25)	66,3%
Корреляция θ на двух параллельных формах одного грейда	0,874

Расхождения сосредоточены у порогов грейда — там, где и эксперт колебался бы. Поэтому результат показывается вместе с **уверенностью** (вероятность, что θ выше порога), а работодатель видит её в профиле. Повторная сдача того же грейда — не раньше чем через 30 дней, смена — не чаще раза в 90 дней: перебором форм «дотянуть» грейд нельзя.

Разделяющая сила заданий

Метрика	Значение
Корреляция «задание — остаток теста» r_pb: среднее · медиана	0,264 · 0,271
Доля заданий с r_pb > 0,2	69%
Индекс D (доля верных у верхних 27% минус у нижних 27%): среднее	0,29
Задания с D ≥ 0,3 (хорошо разделяют) · 0,2–0,3 · < 0,2 (на пересмотр)	47% · 24% · 29% (критерий ≥ 2/3 с D ≥ 0,2 ✓)

Метрика	Значение
d Коэна Middle vs Junior	2,16

Варианты собираются под грейд, поэтому каждое задание видит более узкий диапазон способностей, чем вся популяция: r_pb и D — нижние оценки (сужение диапазона). Задания с D < 0,2 скрипт `item_analysis.py` выводит первыми — их переписывают, перекалибруют или выводят из банка.

Утечка и списывание

Утечка: кандидат уровня Junior (θ = −0,4) заранее знает ответы K «слитых» вариантов теста Middle. Доля сдавших:

K	Единый тест для всех	Генерируемые формы	Формы + детектор + подтверждение
0	7,7%	8,3%	6,7%
1	98,7%	44,3%	**23,0%**
3	98,3%	82,7%	**12,0%**
10	98,3%	97,0%	**1,3%**

Проверка поведения (п. 4а). Модель времени ответа — ожидаемое время задания × логнормальный шум. Уровень Junior (θ ∈ [−0,7; −0,1]) проходит тест Middle: честно, с поиском ответов на стороне (на незнакомых вопросах уходит со вкладки, копирует вопрос, вставляет числовой ответ) или выучив ответы 3 слитых вариантов; ложные срабатывания меряются на честных уровня Middle:

Группа	Риск → подтверждающая форма	Сдали: только детектор утечки	Сдали: + проверка поведения
Честные, уровень Middle	**0,0%**	85,8%	85,8%
Честные, уровень Junior	0,0%	8,0%	8,0%
Ищут ответы на стороне	**99,8%**	51,5%	**0,3%**
Знают слитые ответы	**84,8%**	18,2%	11,7%

Надёжность, калибровка, скорость

Надёжность по длине теста (корреляция θ на параллельных формах): 10 заданий — 0,72; 15 — 0,81; **20 — 0,85**; 25 — 0,86. Поэтому выбрана длина 20 (≈ 10–15 минут).

Калибровка: по 1 500 симулированным прохождениям трудности заданий переоцениваются по ответам; RMSE относительно истинных b падает с 0,30 (экспертные уровни) до 0,17 — банк самоулучшается по мере накопления данных.

Скорость (одно ядро, без БД): генерация индивидуального варианта — 0,2 мс, оценка попытки (EAP, person-fit, детектор утечки) — 3 мс.

Ограничения: симуляция предполагает одномерную способность внутри специализации и модель времени ответа; реальная калибровка и пороги проверки поведения уточняются на пилоте (процедура и скрипты готовы). Полный отчёт: `evaluation/results/tests_report.md`.

8. Регулярные задания (`services/tasks_service.py`)

ТЗ: «система периодически предлагает кандидату короткую задачу от работодателя — решить её или предложить подход». Заказчик уточнил: адресно, формат челленджа на категорию не запрещён, способ оценки — на усмотрение команды.

Форматы. «Решить» — конкретный ответ (число с допуском или короткий текст из списка допустимых), проверяется автоматически сразу после отправки. «Предложить подход» — свободный текст, который работодатель оценивает по рубрике из трёх критериев: корректность, полнота (крайние случаи, ограничения), обоснование компромиссов — по 0–2 балла. Для сортировки откликов система ставит предварительную оценку: долю ключевых идей работодателя, найденных в ответе (по основам слов, RU / EN), и показывает, какие идеи упущены. Ключевые идеи кандидат не видит; итоговая оценка — работодателя.

Кому. «Адресно» — система предлагает задание подходящим кандидатам по одному; «челлендж категории» — открытая задача для всей категории (специализация, при желании грейд), по желанию. Работодатель может и лично предложить задание кандидату из подборки.

Правило предложения. У верифицированного кандидата не больше одного открытого предложения, новое — не чаще раза в 7 дней (`TASK_OFFER_INTERVAL_DAYS`). Задание выбирается по сумме: соответствие грейду (свой грейд — 2, соседний — 0,5, остальные не предлагаются), **работодатель рассматривает кандидата для потребности, к которой привязано задание** (+2 и процент совпадения), задание проверяет компетенцию, которую тест подтвердил слабо (+1 − балл), новизна задания, нагрузка работодателя на проверку (`max_offers`). Кандидат видит, почему ему предложено именно это задание; предложение истекает через срок задания (по умолчанию 7 дней), отклонённое повторно не предлагается. Предложения создаёт фоновый проход раз в час и открытие кандидатом раздела «Задания» или кабинета.

Что даёт результат. Профиль отмечается актуальным (компонент «актуальность» в подборке), на карточке кандидата у работодателя — сигнал «Задания: 3 · средний балл 83%», в объяснении подборки — «Свежий сигнал: задание „Идемпотентность платежей“ — 83%, 3 дня назад». **Грейд задания не меняют**: официально его присваивает только тест.

9. Дизайнеры: как тестировать объективно (концепция)

Заказчик допускает дизайнеров в справочнике, но приоритет — техническим ролям. Механика переносится без изменений (банк → blueprint → IRT → правила грейдов), меняются форматы заданий — только с объективной проверкой:

- **Эвристическая оценка макета:** скриншот интерфейса с заранее внесёнными дефектами (контраст, выравнивание, иерархия, состояния ошибок); кандидат отмечает области — балл по точности и полноте относительно эталонного списка дефектов.
- **Расчётные задания** (параметрические семейства, как у разработчиков): контраст по WCAG для пары цветов, отступы в 8-pt сетке, типографическая шкала, размер области нажатия.
- **Дизайн-системы и доступность:** токены, состояния компонентов, auto-layout, требования WCAG — закрытые вопросы с одним ответом.
- **UX-исследования и метрики:** интерпретация A/B-теста, расчёт SUS, воронка конверсии — числовые ответы.
- **Портфолио** — экспертная рубрика как атрибут профиля, не влияющий на грейд (субъективно).

Компетенции специализации «UI/UX-дизайн»: визуальный дизайн, UX и исследования, дизайн-системы, доступность, прототипирование — с теми же квотами в blueprint.

6. Интеграция с реестром ФСП

API реестра ФСП на хакатоне не предоставляется, поэтому реализованы **порт** и два **адаптера**: платформа зависит только от интерфейса, замена мока на реальную интеграцию не затрагивает бизнес-логику.

```
class FSPRegistry(Protocol):              # backend/app/services/fsp/registry.py
    source: str
    def get_participant(self, fsp_id: str) -> RegistryParticipant | None: ...
    def find_by_email(self, email: str) -> RegistryParticipant | None: ...
```

Адаптер	Режим (`FSP_REGISTRY_MODE`)	Описание
`MockFSPRegistry`	`mock` (по умолчанию)	Синтетический снимок реестра `mock_data/fsp_registry.json`
`HttpFSPRegistry`	`http`	REST-клиент; контракт ниже. Мок-API поднят самим backend: `/api/mock-fsp/v1`

Контракт REST API реестра

```
GET /participants/{fsp_id}          → 200 Participant | 404
GET /participants?email=<e-mail>    → 200 Participant | 404
```

```
{
  "fsp_id": "FSP-100777",
  "full_name": "Демидов Алексей",
  "email": "candidate@demo.ru",
  "region": "г. Москва",
  "sport_rank": "I разряд",
  "achievements": [{
    "id": "ACH-2025-31415",
    "event_name": "Всероссийские соревнования по продуктовому программированию",
    "discipline": "Продуктовое программирование",
    "level": "all_russian",            // regional | interregional | all_russian | international
    "date": "2026-05-08",
    "place": 2, "participants_total": 180,
    "team_name": "Byte Busters", "role": "backend",
    "score": 91.5, "specialization_hint": "backend",
    "certificate_url": "https://registry.fsp.example/certificates/ACH-2025-31415"
  }]
}
```

Набор полей выбран так, чтобы работодатель понимал **уровень** (тип соревнования, место среди скольких участников), **релевантность** (дисциплина, роль в команде) и **проверяемость** (ссылка на сертификат). Дисциплины соответствуют направлениям ФСП: продуктовое, алгоритмическое программирование, информационная безопасность, программирование БАС и робототехники.

Связь профиля с ФСП ID

1. **Автоматически**: после подтверждения e-mail кандидата выполняется поиск в реестре по e-mail (`fsp_service.try_auto_link`). Найден — профиль обогащается без действий пользователя.
2. **Вручную**: кандидат вводит ФСП ID. Если e-mail в реестре совпадает — привязка мгновенная; иначе код подтверждения отправляется на e-mail из реестра (владение подтверждается, чужие достижения присвоить нельзя). Один ФСП ID — один профиль (UNIQUE + проверка, 409).

3. **Целевое состояние**: вход «через ФСП ID» на Keycloak. Токен Keycloak содержит `sub` участника — он сохраняется в `users.external_subject`, шаг с кодом заменяется OIDC-авторизацией.

Данные кешируются локально (`fsp_profiles`, `fsp_achievements`) с отметкой `synced_at` и источником; кнопка «Синхронизировать» перечитывает реестр. Отвязка удаляет локальный снимок.

Как ФСП влияет на ранжирование

Сила сигнала для специализации `s` (`services/fsp/scoring.py`):

```
points(a) = level_weight(a.level) × place_factor(a.place, a.participants_total)
          × recency(0.5 ^ (years / 2.5)) × relevance(a.discipline, a.role, s)
strength  = 1 − exp(−(Σ points + rank_bonus) / 1.2)          ∈ [0, 1)
```

- `level_weight`: международные 1.0, всероссийские 0.85, межрегиональные 0.6, региональные 0.45;
- `place_factor`: 1 место 1.0, 2 — 0.85, 3 — 0.75, топ-10% — 0.55, топ-30% — 0.4, участие — 0.25;
- `relevance`: роль в команде совпадает со специализацией — 1.0, иначе матрица «дисциплина × специализация» (например, ИБ → DevOps 0.9, алгоритмическое → Backend 0.8). Заказчик уточнил: все дисциплины ФСП одинаково приоритетны, но учитывать актуальность направления разумно — специалист по ИБ с соревнованиями по ИБ релевантнее, чем с робототехникой. Поэтому базовые веса от дисциплины не зависят, а близость дисциплины к специализации потребности умножает вклад достижения и показывается работодателю: «профильная дисциплина» (≥ 0,75), «смежная» (≥ 0,45), «другое направление»;
- `rank_bonus`: КМС/МС/разряды.

В Matching Engine компонент ФСП = `0.3 + 0.7 × strength`. **Отсутствие истории ФСП ≠ 0**: используется нейтральная база 0.3, сигнал считается «недоступным», и кандидат не штрафуется; любое участие может только поднять профиль. Валидация: идеальные кандидаты без ФСП в среднем находятся на 4,5-м перцентиле выдачи против 3,6-го у кандидатов с ФСП (см. matching.md). Кандидат может скрыть ФСП в настройках — тогда сигнал не используется и в ранжировании.

UI

- Кандидат: страница «Достижения ФСП» — список с медалями, уровнем, местом, командой, ролью; сила сигнала; привязка/синхронизация/отвязка; явное сообщение «История ФСП не найдена — это нормально».
- Работодатель: бейдж «N достижений ФСП» в карточке, лучшая позиция в объяснении («2 место — … · профильная дисциплина»), полный список в профиле с отметкой профильности каждого достижения для специализации потребности; фильтр «есть / нет достижений ФСП» в подборке и банке кандидатов.

7. API

Полная интерактивная спецификация OpenAPI 3.1 (Swagger UI): **http://localhost:8000/api/docs** (ReDoc: `/api/redoc`, JSON: `/api/openapi.json`). Все запросы, ответы и коды ошибок описаны в спецификации; TypeScript-типы фронтенда генерируются из неё (`npm run gen:api`). Корень backend (`http://localhost:8000/`, а также `/docs`, `/redoc`, `/openapi.json`) перенаправляет на документацию, `GET /api` возвращает ссылки на неё и на `/api/health`.

- Аутентификация: `Authorization: Bearer <JWT>` — токен из `/auth/login` или `/auth/verify-email`.
- Формат ошибок: `{"code": "cooldown", "message": "…", "details": {...}}` — в том числе для несуществующего адреса (`route_not_found`, 404) и неподдерживаемого метода (`method_not_allowed`, 405).
- Язык: заголовок `Accept-Language` (`ru` — по умолчанию, `en`). На нём возвращаются сообщения об ошибках (включая 422 с названием поля), объяснения подбора, рекомендации по тесту, справочники и тексты заданий; ответ содержит `Content-Language` и `Vary: Accept-Language`. `code` ошибки от языка не зависит.
- Коды: `401` нет/просрочен токен, `403` чужая роль, `404` не найдено или нет доступа, `409` конфликт состояния (кулдаун, дубликат, закрытое приглашение), `422` валидация, `429` лимит приглашений.

Эндпоинты

Аутентификация

Метод	Путь	Назначение
POST	`/api/auth/register`	Регистрация (роль, согласие 152-ФЗ) → код на e-mail
POST	`/api/auth/verify-email`	Подтверждение кода → JWT
POST	`/api/auth/resend-code`	Повторная отправка кода
POST	`/api/auth/login`	Вход → JWT
GET	`/api/auth/me`	Текущий пользователь (включая `locale` — язык интерфейса)
PUT	`/api/auth/me/locale`	Сохранить язык интерфейса в учётной записи (`ru` / `en`)

Справочники

GET	`/api/reference`	Специализации и компетенции, грейды с порогами, навыки, профили весов

Кабинет кандидата

Метод	Путь	Назначение
GET	`/api/candidate/dashboard`	Категория, заполненность, шаги, приглашения, последний тест, предложенное задание
GET/PUT	`/api/candidate/profile`	Профиль, резюме, опрос (специализация, грейд), навыки, `experience` — места работы (список заменяется целиком); `categories` — все подтверждённые категории, `grade_status` — статус заявленного грейда
PUT	`/api/candidate/categories/{specialization_id}/primary`	Сделать категорию основной

Метод	Путь	Назначение
POST	`/api/candidate/resume/parse`	Распознавание PDF-резюме (без сохранения): контакты, стек, грейд, стаж, места работы
GET/PUT	`/api/candidate/settings`	Приватность и согласие на публикацию
GET	`/api/candidate/fsp`	Достижения ФСП и сила сигнала
POST	`/api/candidate/fsp/link` · `/fsp/confirm` · `/fsp/sync`	Привязка / подтверждение / синхронизация ФСП ID
DELETE	`/api/candidate/fsp/link`	Отвязка
GET	`/api/candidate/invitations`	Входящие приглашения
GET	`/api/candidate/invitations/{id}`	Открыть (статус → `viewed`)
POST	`/api/candidate/invitations/{id}/accept` · `/reject` · `/report`	Ответ / жалоба
GET	`/api/candidate/tasks`	Регулярные задания: адресные предложения (с причинами и сроком), челленджи категории, мои результаты; при необходимости предлагает следующее
POST	`/api/candidate/tasks/{id}/submit`	Ответ: автопроверка (`correct`) или подход на проверку работодателю
POST	`/api/candidate/tasks/offers/{assignment_id}/decline`	Отказаться от предложенного задания

Тестирование

Метод	Путь	Назначение
GET	`/api/candidate/testing?specialization_id=`	Доступность по грейдам (кулдауны категории этой специализации), категории, активная попытка, история, правила
POST	`/api/candidate/testing/start`	Сгенерировать индивидуальный вариант `{specialization_id, target_grade}`
GET	`/api/candidate/testing/attempts/{id}`	Вопросы попытки (без правильных ответов)
POST	`/api/candidate/testing/answer`	Сохранить ответ `{attempt_id, item_id, answer: [...], time_ms?, pasted?, client?: {focus_lost, away_ms, copies}}`
POST	`/api/candidate/testing/finish`	Завершить `{attempt_id, client?}` → грейд, категория, компетенции, уверенность, `integrity` (проверка честности)
GET	`/api/candidate/testing/results/{attempt_id}`	Результат

Кабинет работодателя

Метод	Путь	Назначение
GET	`/api/employer/dashboard`	Потребности, воронка приглашений, последние ответы
GET/PUT	`/api/employer/profile`	Профиль компании

Метод	Путь	Назначение
GET/POST	`/api/employer/requirements`	Потребности; создание сразу формирует подборку
POST	`/api/employer/requirements/parse`	Текст потребности → специализация, грейд, стек, вилка
GET/PUT/DELETE	`/api/employer/requirements/{id}`	Потребность: просмотр / уточнение (пересчёт) / закрытие
GET	`/api/employer/requirements/{id}/matches`	Подборка: `grades, categories, skills, has_fsp, min_score, shortlisted, refresh`
POST	`/api/employer/requirements/{id}/shortlist/{candidate_id}?value=`	Избранное
GET	`/api/employer/requirements/{id}/compare?a=&b=`	Почему A выше B
GET	`/api/employer/candidates`	Банк кандидатов с фильтрами
GET	`/api/employer/candidates/{id}?requirement_id=`	Профиль + объяснение; контакты — по правилам видимости
GET/POST	`/api/employer/invitations`	История / отправка приглашения (зарплата обязательна)
POST	`/api/employer/invitations/{id}/withdraw`	Отозвать
GET	`/api/employer/applications` · POST `/applications/{id}/status`	Отклики на вакансии
GET/POST	`/api/employer/tasks`	Задания компании со статистикой / публикация: `mode` (solve / approach), `audience` (targeted / category), `grade_id`, `job_id`, `competency_id`, `answer_spec`, `key_points`, `time_limit_days`, `max_offers`
PUT	`/api/employer/tasks/{id}/active`	Снять с публикации / вернуть
POST	`/api/employer/tasks/{id}/offer`	Предложить задание кандидату лично `{candidate_id}`
GET	`/api/employer/tasks/{id}/submissions`	Решения: автопроверка или предварительная оценка по ключевым идеям (что найдено / упущено)
POST	`/api/employer/submissions/{id}/review`	Оценка по рубрике `{rubric: {correctness, completeness, reasoning: 0..2}, comment}`

Вакансии и отклики

GET	`/api/jobs`	Опубликованные вакансии (для кандидата — с % совпадения, обратный матчинг)
GET	`/api/jobs/{id}`	Вакансия

GET	/api/jobs	Опубликованные вакансии (для кандидата — с % совпадения, обратный матчинг)
POST	/api/applications	Отклик кандидата (раскрывает контакты этому работодателю)
GET	/api/applications	Мои отклики

Мок реестра ФСП

GET	/api/mock-fsp/v1/participants/{fsp_id}	Участник
GET	/api/mock-fsp/v1/participants?email=	Поиск по e-mail
GET	/api/mock-fsp/v1/stats	Размер мок-реестра

Пример: потребность → подборка → приглашение

```
TOKEN=$(curl -s -X POST localhost:8000/api/auth/login -H 'Content-Type: application/json' \
  -d '{"email":"hr@demo.ru","password":"demo12345"}' | jq -r .access_token)

curl -s -X POST localhost:8000/api/employer/requirements -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"title":"Backend Developer","specialization_id":"backend",
  "grade_id":"middle","must_skills":["python","fastapi","postgresql","docker"],
  "salary_from":180000,"salary_to":250000}' | jq .id

curl -s "localhost:8000/api/employer/requirements/13/matches?limit=3" -H "Authorization: Bearer $TOKEN" \
  | jq '.items[] | {name: .display_name, category: .category.label, percent: .match.percent,
        why: [.match.reasons[].text]}'
```

Фрагмент ответа:

```
{
  "name": "Алексей Демидов", "category": "Backend · Middle", "percent": 93,
  "why": ["Специализация Backend совпадает", "Подтверждённый грейд Middle — как требуется",
          "Совместимость стека 90%: 4 из 4 обязательных навыков, 3 подтверждены тестом",
          "Тест: 76/100 (уверенность 100%)",
          "3 подтверждённых достижения ФСП: 2 место — Всероссийские соревнования по продуктовому
программированию (2026)"]
}
```

8. Сборка и развёртывание

Вариант 1. Docker Compose (рекомендуется)

Требования: Docker 24+ с Compose v2, ~1 ГБ свободной памяти.

```
cp .env.example .env        # необязательно: значения по умолчанию работают
docker compose up --build
```

Сервис	Адрес	Что это
frontend	http://localhost:8080	Веб-приложение (nginx, проксирует `/api` в backend)
backend	http://localhost:8000/api/docs	API + Swagger UI
mailpit	http://localhost:8025	Все письма платформы (коды подтверждения)
db	внутренняя сеть	PostgreSQL 16, том `pgdata`

При старте backend ждёт БД, применяет миграции Alembic и **один раз** заполняет пустую базу демо-данными (`python -m app.seed --if-empty`). Повторный запуск данные не трогает.

Полезные команды:

```
docker compose logs -f backend                                  # логи API
docker compose exec backend python -m app.seed --reset          # пересоздать демо-данные
docker compose exec backend pytest -q                           # тесты внутри контейнера
docker compose down -v                                          # остановить и удалить данные
python scripts/smoke_test.py --base http://localhost:8080       # сквозной smoke-тест по HTTP
```

Вариант 2. Локально без Docker

Требования: Python 3.12, Node.js 20+. По умолчанию используется SQLite (`backend/fsp_talent.db`), PostgreSQL не нужен.

```
# Backend
cd backend
python -m venv .venv
.venv/Scripts/activate            # Windows;  source .venv/bin/activate — Linux/macOS
pip install -r requirements-dev.txt
alembic upgrade head
python -m app.seed                # демо-данные (можно с --reset)
uvicorn app.main:app --port 8000 --reload

# Frontend (в другом терминале)
cd frontend
npm ci
npm run dev                        # http://127.0.0.1:5173, /api проксируется на :8000
```

Чтобы работать с PostgreSQL локально, задайте
`DATABASE_URL=postgresql+psycopg://fsp:fsp_password@localhost:5432/fsp_talent` .

Проверки качества

```
cd backend && pytest -q && ruff check app tests scripts      # 446 тестов, линтер
cd frontend && npm test                  # 124 теста: i18n, API-клиент, язык, страницы, задания, опыт
cd frontend && npm run typecheck && npm run lint && npm run build
python evaluation/evaluate_tests.py      # валидация механики тестирования (~2 мин)
python evaluation/evaluate_matching.py   # бенчмарк подбора (~30 с)
python evaluation/tune_matching.py       # подбор гиперпараметров (~4 мин)
```

Скрипты оценки не требуют БД и сервера — используют те же модули движков, что и API. Результаты сохраняются в `evaluation/results/*.md|json`.

Переменные окружения

Полный список с комментариями — в `.env.example`. Основные:

Переменная	По умолчанию	Назначение
`DATABASE_URL`	SQLite в `backend/`	Строка подключения SQLAlchemy
`JWT_SECRET`	dev-значение	**Обязательно сменить в продакшене**
`EMAIL_DEV_EXPOSE_CODE`	`true`	Демо-режим: код подтверждения виден в UI. В продакшене — `false`
`SMTP_HOST` / `SMTP_PORT`	— / 1025	SMTP (в compose — Mailpit)
`TEST_LENGTH`, `TEST_TIME_LIMIT_MINUTES`	20, 30	Параметры теста
`RETAKE_COOLDOWN_DAYS`, `GRADE_CHANGE_COOLDOWN_DAYS`	30, 90	Кулдауны
`MAX_CATEGORIES`	3	Сколько подтверждённых категорий может быть у кандидата
`TASK_OFFER_INTERVAL_DAYS`, `TASK_OFFER_LOOP_MINUTES`	7, 60	Регулярные задания: не чаще раза в N дней на кандидата; фоновый проход (0 — только по запросу)
`FSP_REGISTRY_MODE`, `FSP_REGISTRY_URL`	`mock`	Источник данных ФСП: `mock` или `http`
`MATCHING_WEIGHTS_JSON`	—	Переопределение весов матчинга
`INVITATIONS_DAILY_LIMIT`	40	Анти-спам лимит приглашений

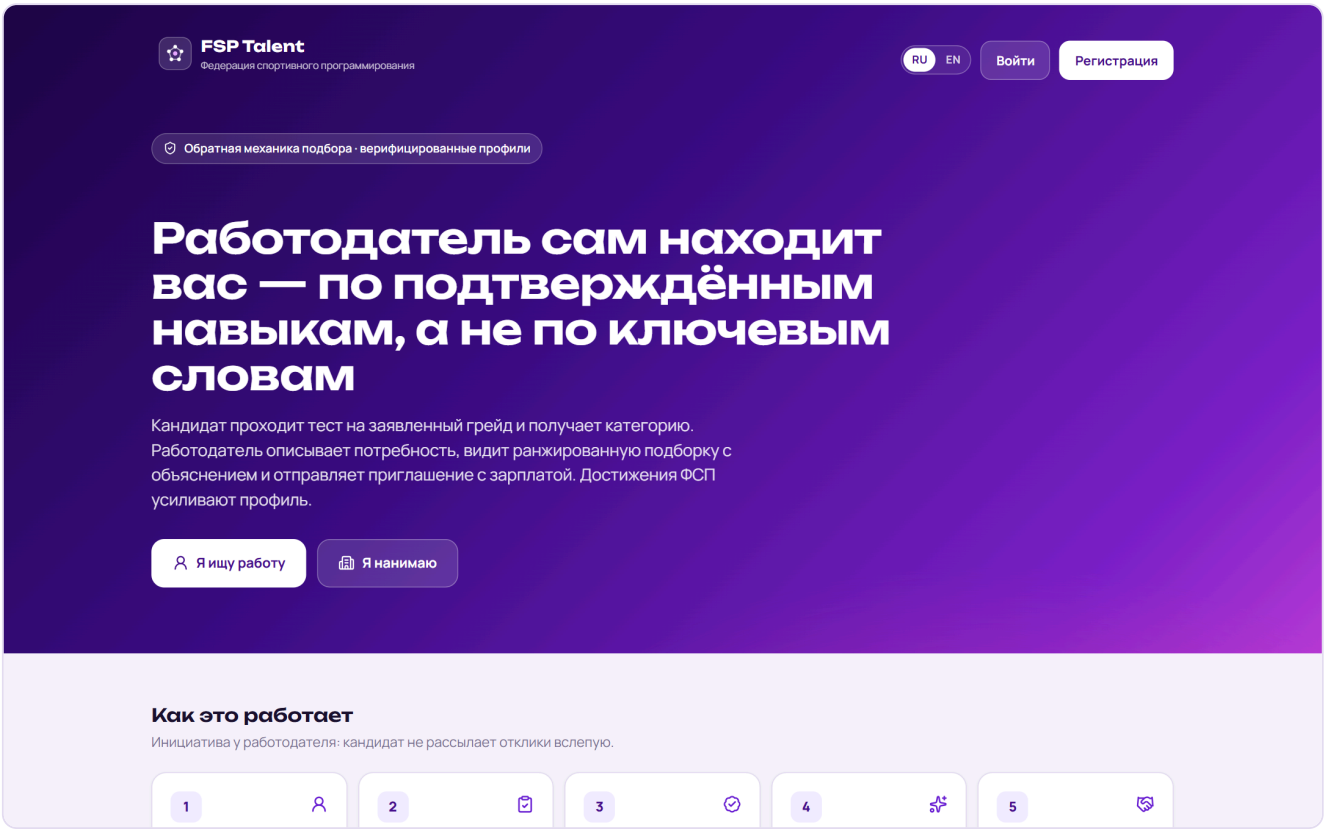
Развёртывание на сервере

Достаточно одной ВМ (2 vCPU / 2 ГБ): `docker compose up -d --build`, затем поставить перед портом 8080 TLS-терминатор (nginx/Caddy/Traefik) и закрыть внешний доступ к 8000/8025. Перед публичным запуском: `JWT_SECRET`, `POSTGRES_PASSWORD`, `EMAIL_DEV_EXPOSE_CODE=false`, реальный SMTP.

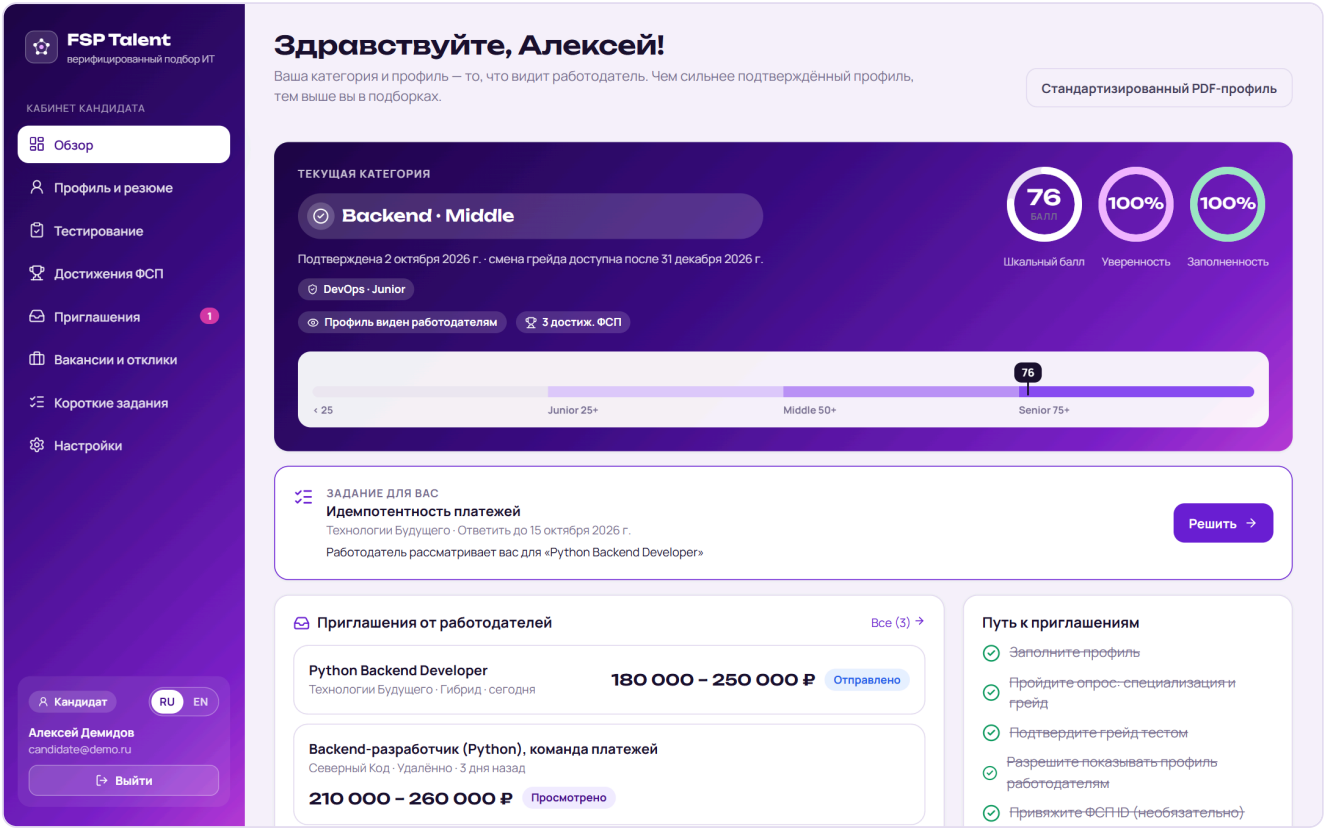
Типовые проблемы

- **Порт занят** (8000/8080/8025) — измените публикацию портов в `docker-compose.yml`.
- **Vite не открывается по `localhost`** на Windows — используйте `http://127.0.0.1:5173` (dev-сервер слушает IPv4).
- **Пустая выдача у работодателя** — кандидат попадает в подборки только после подтверждённой категории и согласия на публикацию профиля (Настройки → «Публикация профиля»).

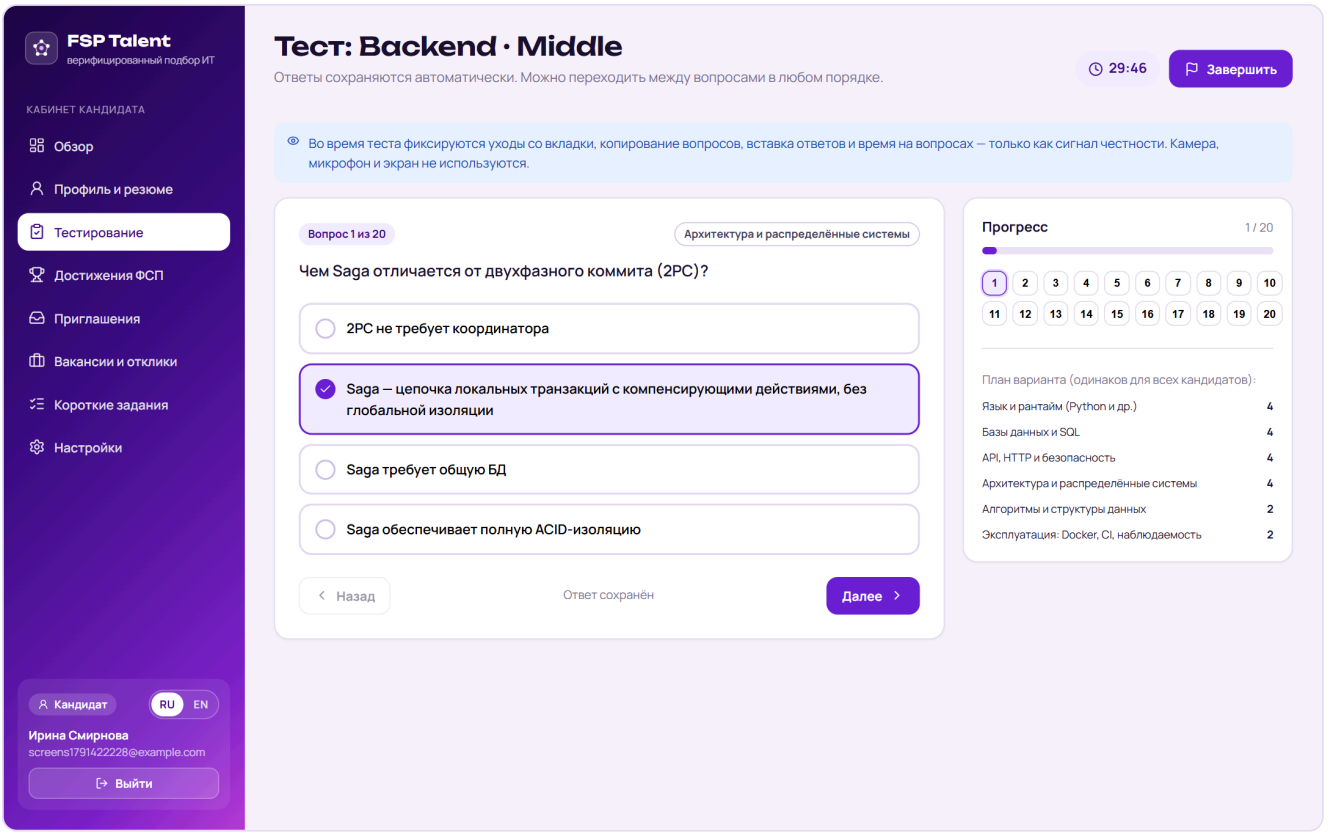
9. Интерфейс



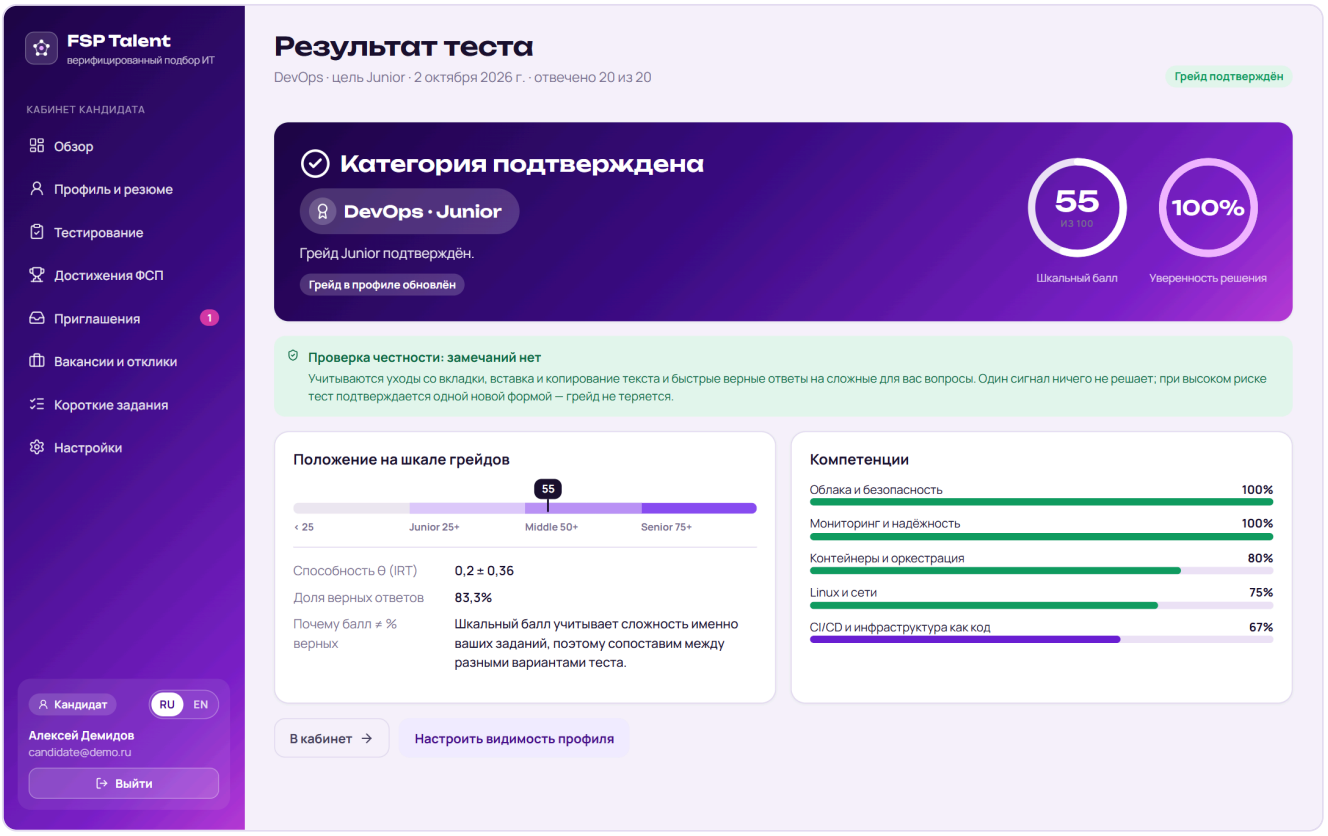
Главная страница: обратная механика подбора и демо-доступ



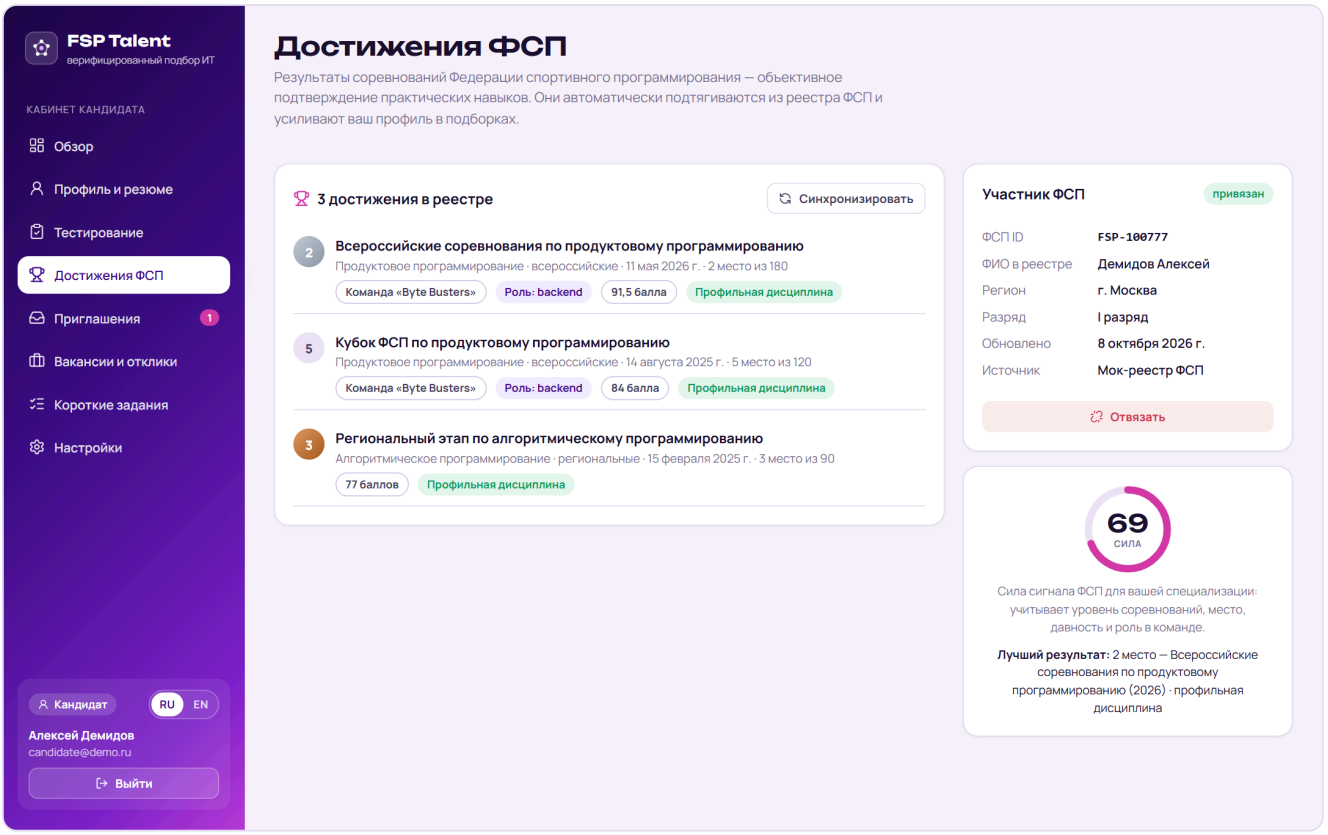
Кабинет кандидата: подтверждённая категория, шкальный балл, приглашения



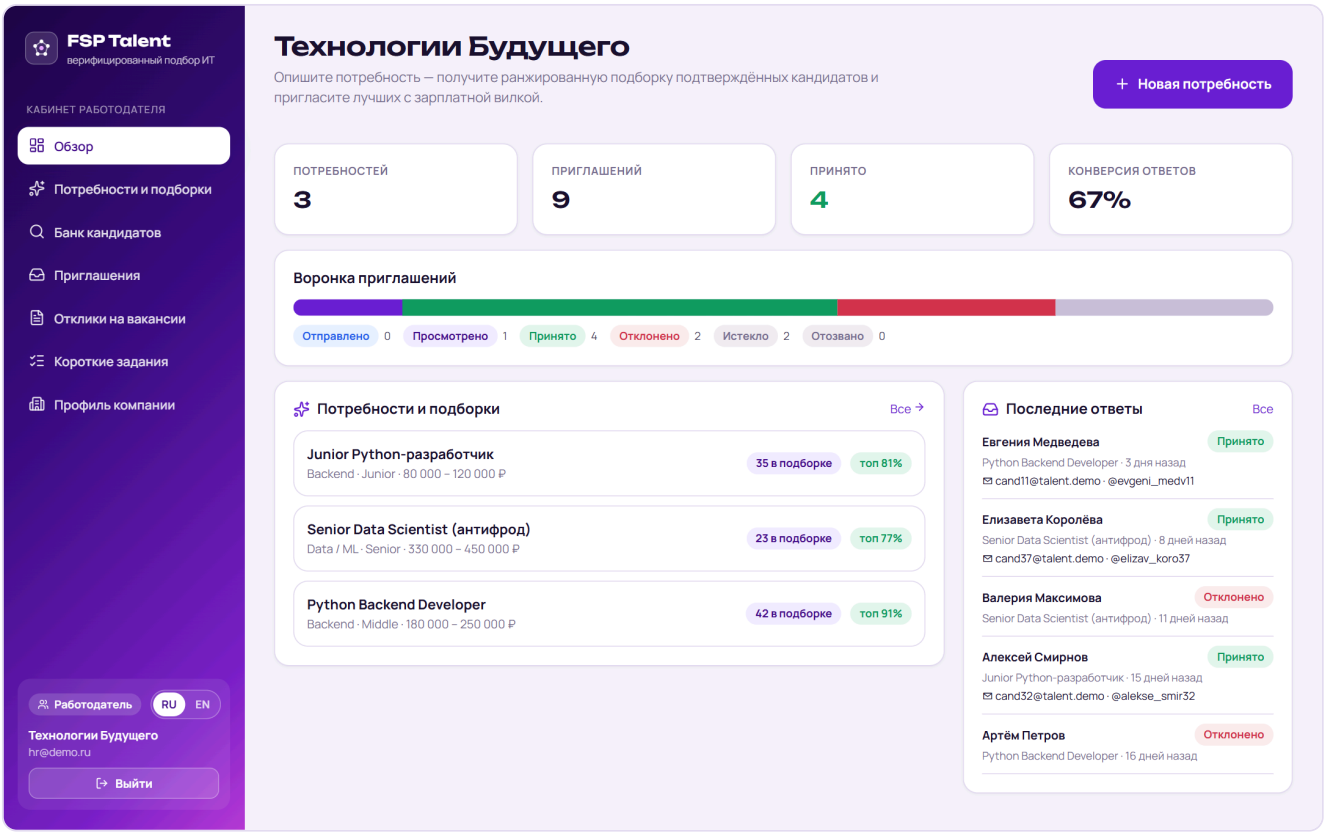
Прохождение теста: индивидуальный вариант по единому плану, таймер, автосохранение



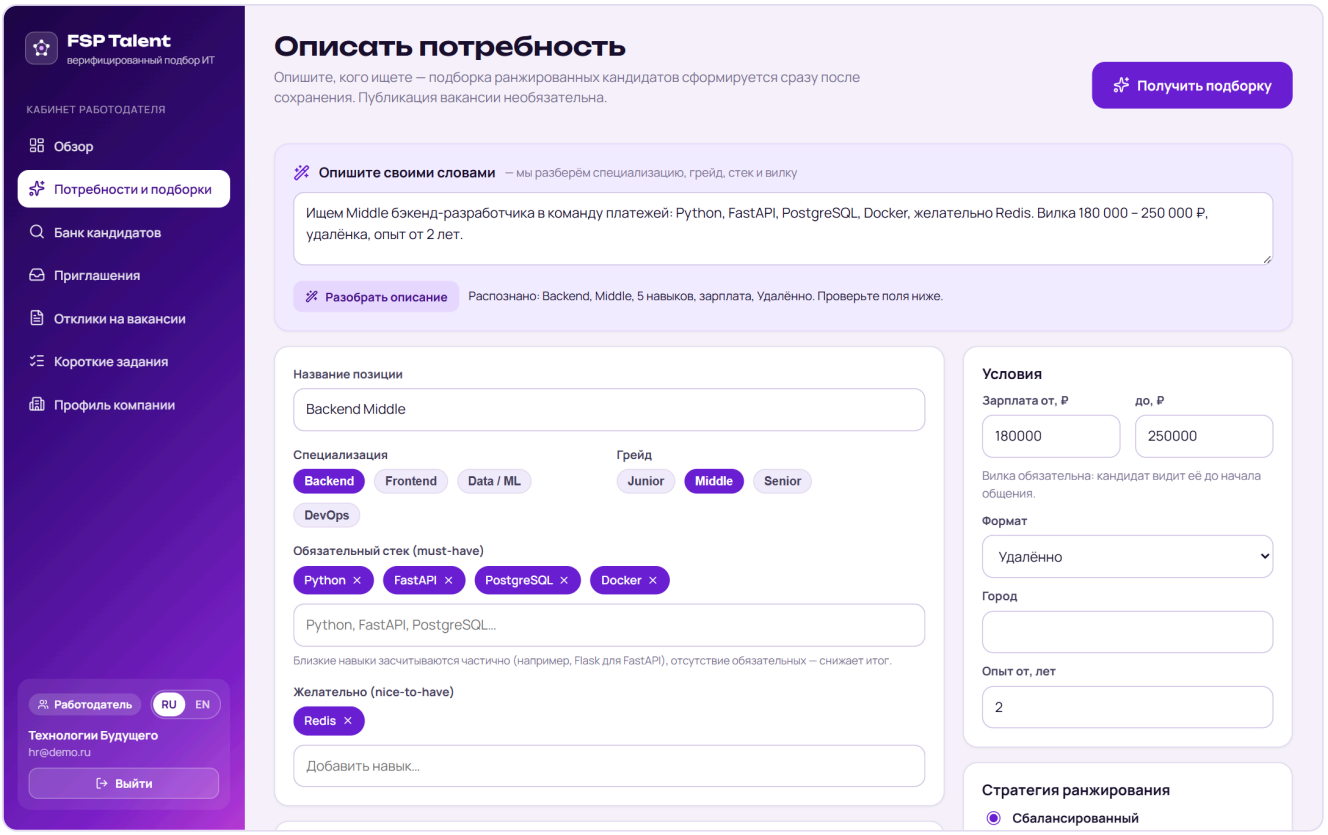
Результат теста: категория, положение на шкале грейдов, компетенции, уверенность



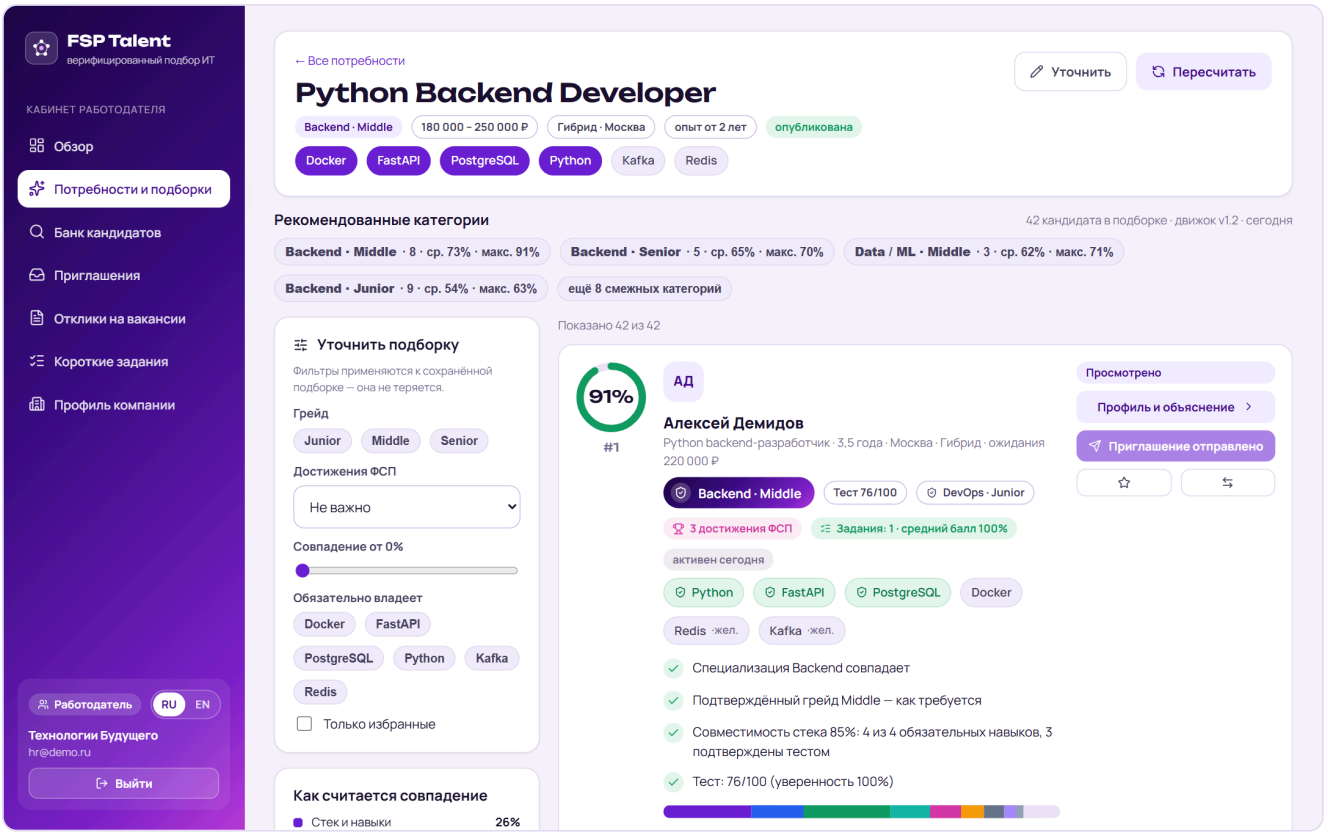
Достижения ФСП: автоматическое обогащение профиля из реестра



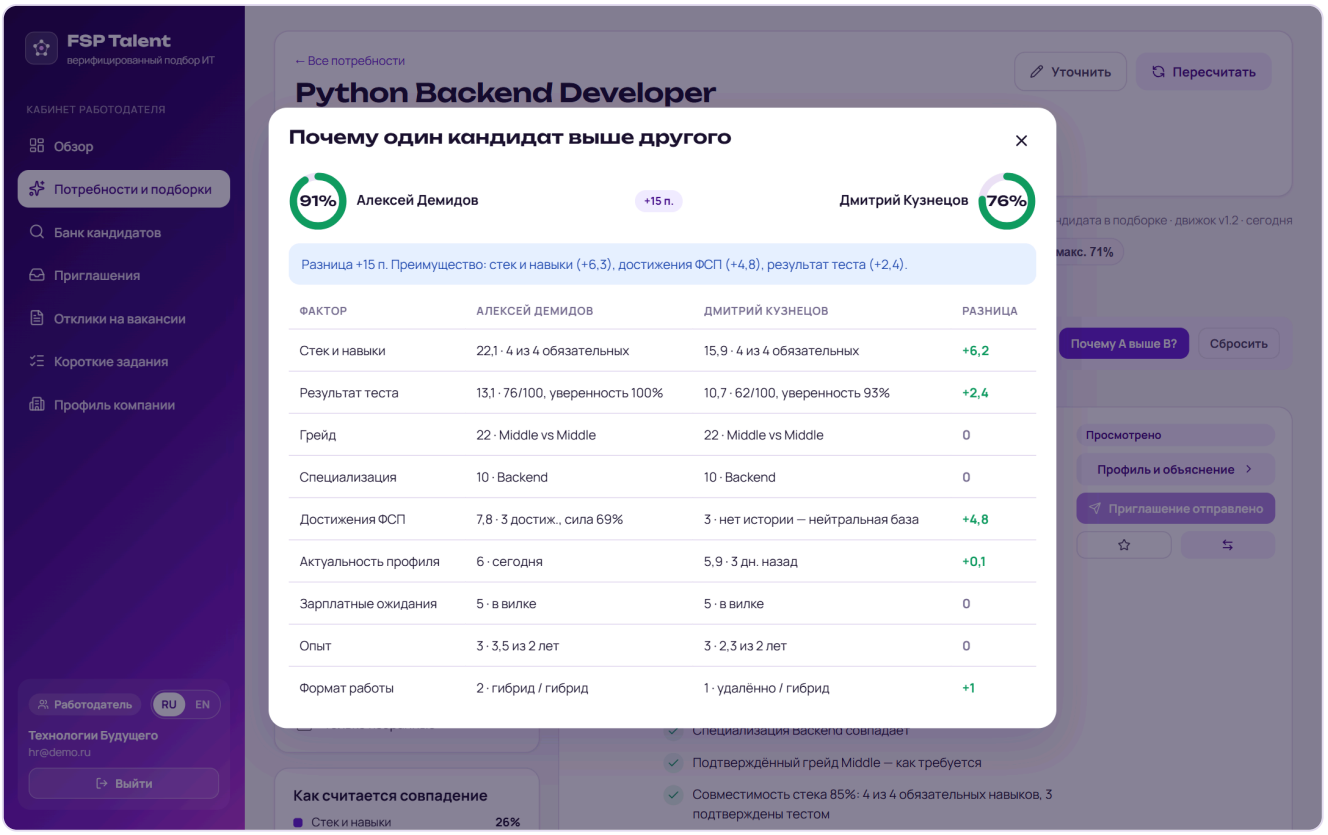
Кабинет работодателя: потребности, воронка приглашений



Описание потребности: разбор текста в специализацию, грейд, стек и вилку



Подборка: рекомендованные категории, рейтинг, объяснение, вклад факторов



Почему кандидат А выше кандидата В

FSP Talent
верифицированный подбор ИТ

КАБИНЕТ РАБОТОДАТЕЛЯ

- Обзор
- Потребности и подборки
- Банк кандидатов
- Приглашения
- Отклики на вакансии
- Короткие задания
- Профиль компании

Работодатель · RU · EN

Технологии Будущего
hr@demo.ru

Выйти

← К подборке «Python Backend Developer»

АД

Алексей Демидов

Python backend-разработчик · 3,5 года опыта · Москва · Гибрид

Backend · Middle · DevOps · Junior · 3 достижения ФСП · 2 призовых · Задания: 1 · средний балл 100%

91%
совпадение с потребностью

Приглашение отправлено

Ожидания: 220 000 ₽

Почему этот кандидат в подборке

для «Python Backend Developer»

- Специализация Backend совпадает
- Подтверждённый грейд Middle — как требуется
- Совместимость стека 85%: 4 из 4 обязательных навыков, 3 подтверждены тестом
- Тест: 76/100 (уверенность 100%)
- 3 подтверждённых достижения ФСП: 2 место — Всероссийские соревнования по продуктовому программированию (2026) · профильная дисциплина
- Профиль активен сегодня
- Свежий сигнал: задание «Пропускная способность пула воркеров» — 100%, 11 дней назад
- Ожидания 220 000 ₽ — в вилке

Стек: обязательные и желательные навыки

Python · FastAPI · PostgreSQL · Docker · Redis ·жел. · Kafka ·жел.

Зелёный — подтверждён тестом · жёлтый — близкий навык засчитан частично · красный — отсутствует

Вклад факторов в 91%

Стек и навыки 22,1 · Результат теста 13,1 · Грейд 22 · Специализация 10 · Достижения ФСП 7,8 · Актуальность профиля 6 · Зарплатные ожидания 5 · Опыт 3 · Формат работы 2

ФАКТОР	ОЦЕНКА	ВЕС	ВКЛАД	ДЕТАЛИ
Стек и навыки	85	26%	22,1	4 из 4 обязательных
Результат теста	82	16%	13,1	76/100, уверенность 100%
Грейд	100	22%	22	Middle vs Middle
Специализация	100	10%	10	Backend
Достижения ФСП	78	10%	7,8	3 достиж., сила 69%
Актуальность профиля	100	6%	6	сегодня
Зарплатные ожидания	100	5%	5	в вилке
Опыт	100	3%	3	3,5 из 2 лет
Формат работы	100	2%	2	гибрид / гибрид

Подтверждённый уровень

тест от 2 октября 2026 г.

76

‹ 25 · Junior 25+ · Middle 50+ · Senior 75+

Компетенции

- Алгоритмы и структуры данных — 100%
- API, HTTP и безопасность — 100%
- Эксплуатация: Docker, CI, наблюдаемость — 50%
- Архитектура и распределённые системы — 38%
- Язык и рантайм (Python и др.) — 100%
- Базы данных и SQL — 100%

История тестирования

DevOps · Junior	55	Грейд подтверждён
Backend · Middle	76	Грейд подтверждён

Достижения ФСП

2 Всероссийские соревнования по продуктовому программированию
Продуктовое программирование · всероссийские · 11 мая 2026 г. · 2 место из 180
Команда «Byte Busters» · Роль: backend · 91,5 балла · Профильная дисциплина

5 Кубок ФСП по продуктовому программированию
Продуктовое программирование · всероссийские · 14 августа 2025 г. · 5 место из 120
Команда «Byte Busters» · Роль: backend · 84 балла · Профильная дисциплина

Контакты

Скрыты до тех пор, пока кандидат не примет ваше приглашение или не откликнется сам. Так кандидат видит ваши условия до начала общения.

Предложить задание

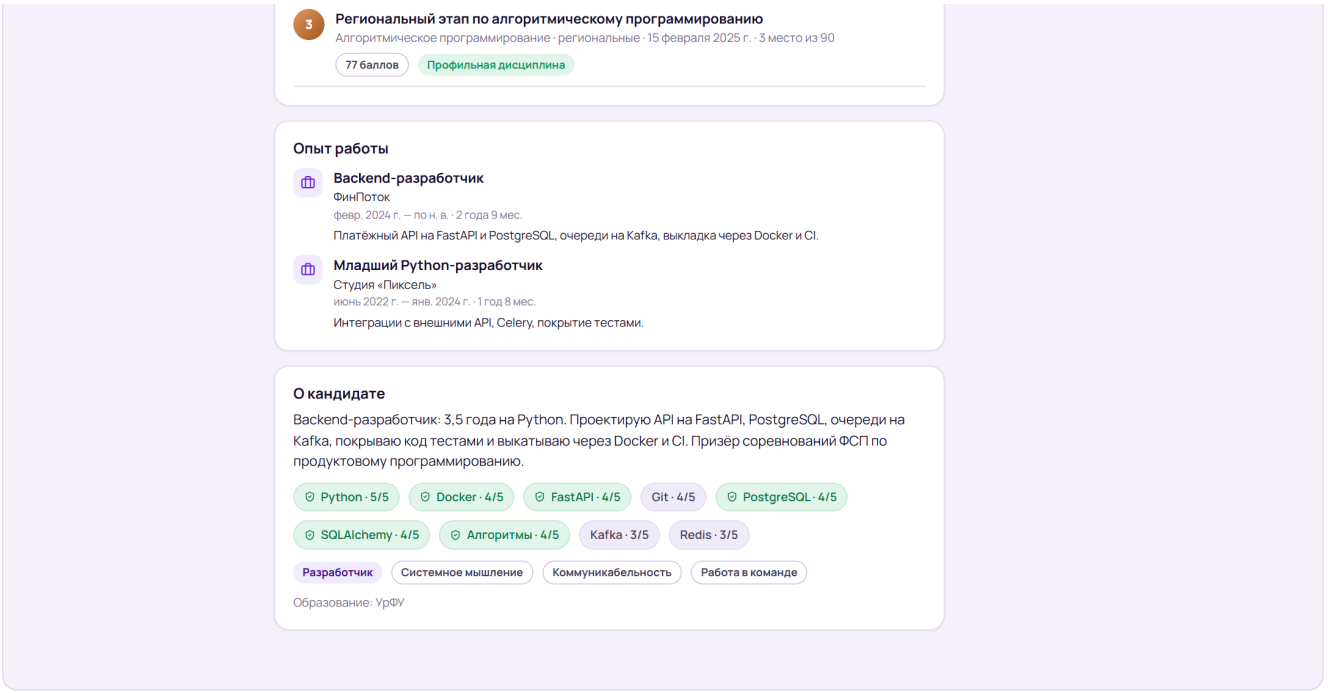
Идемпотентность платежей

Кандидат увидит его в разделе «Задания» с пометкой «лично от работодателя».

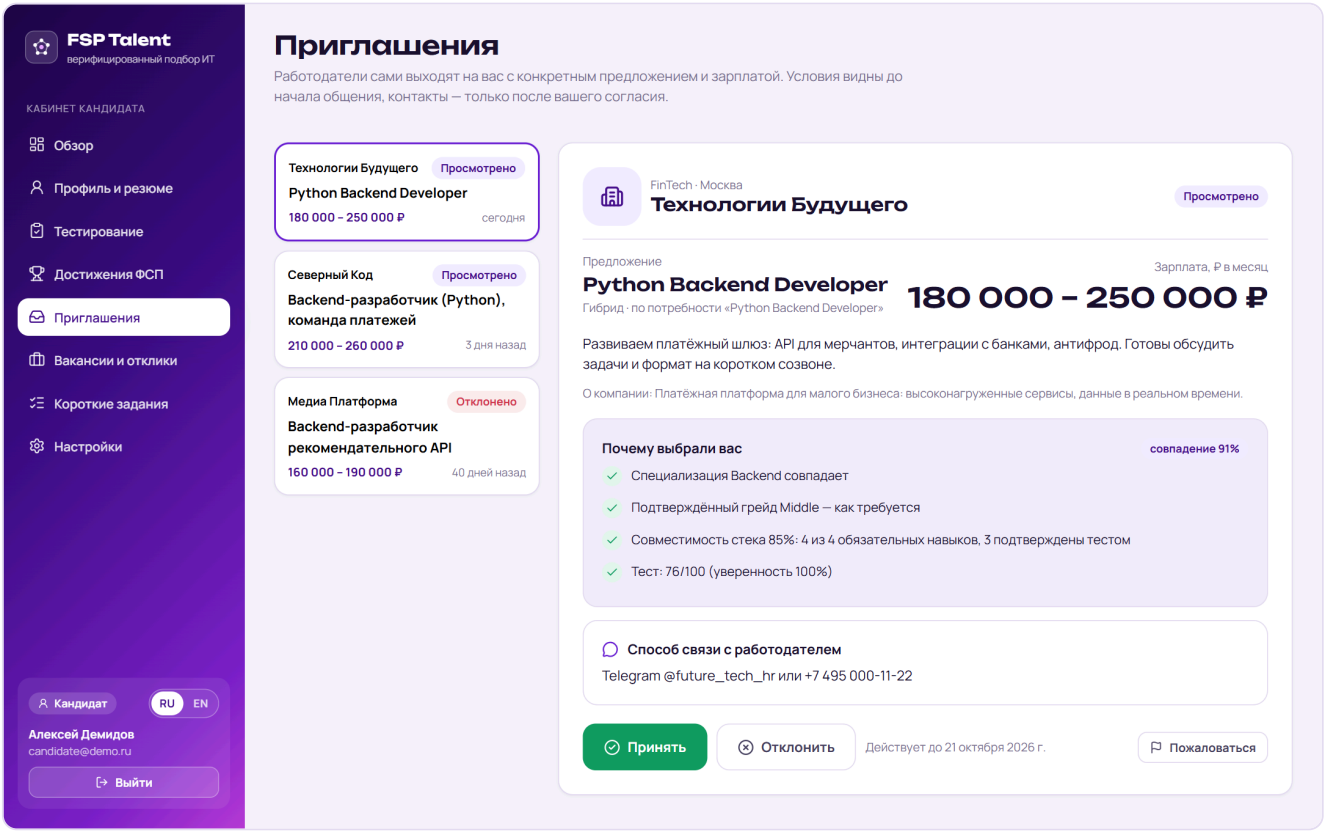
Предложить задание

Ваши приглашения

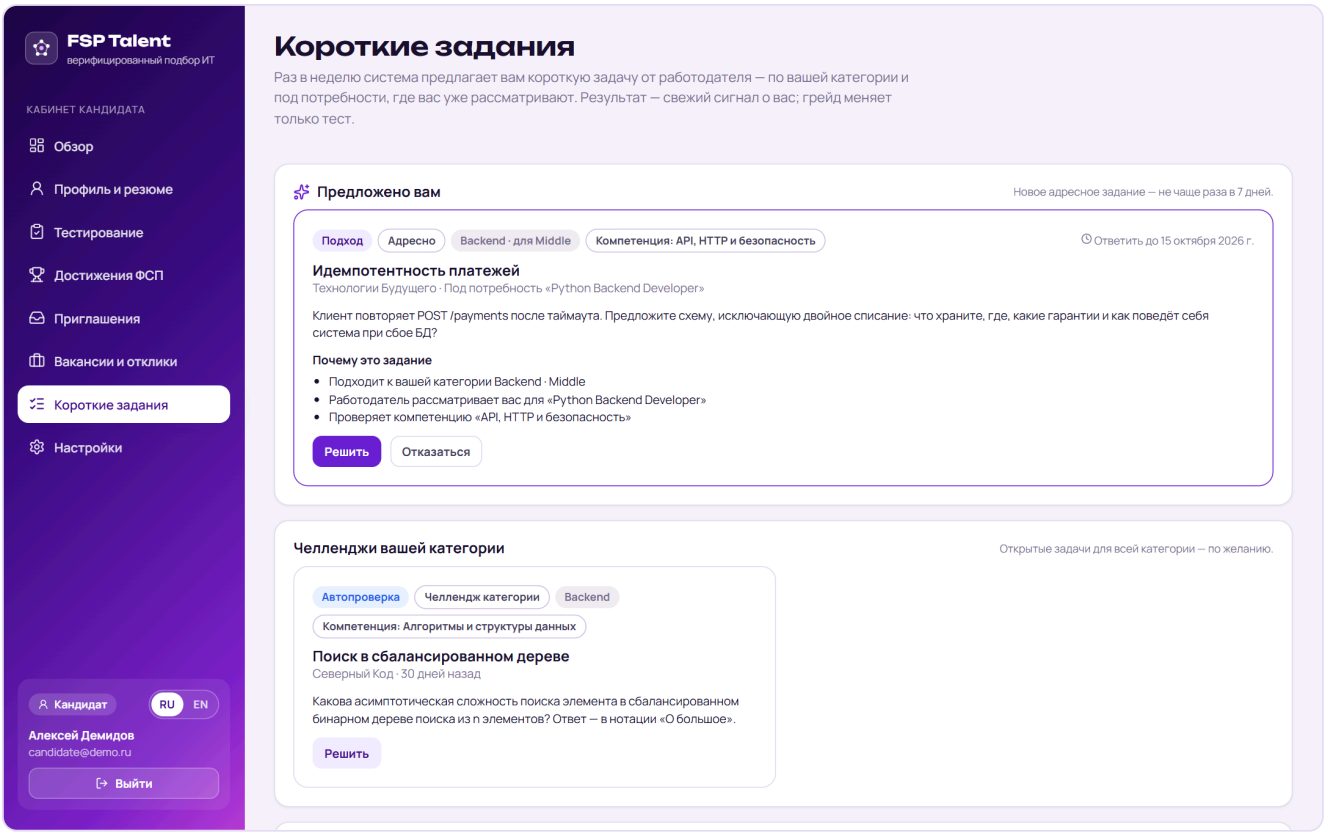
Python Backend Developer · Просмотрено
180 000 – 250 000 ₽ · сегодня



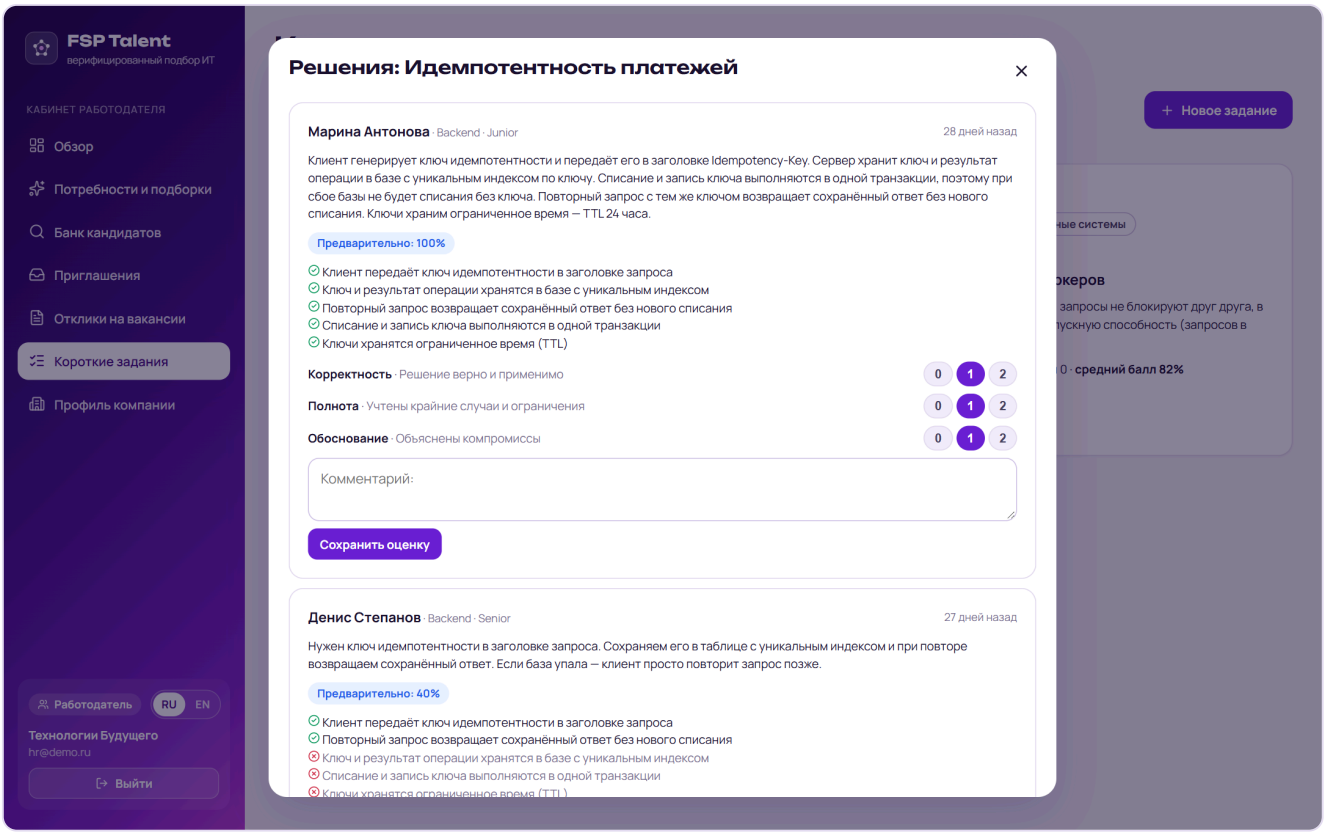
Профиль кандидата для работодателя: объяснение совпадения, контакты скрыты



Приглашение у кандидата: компания, предложение и зарплата до ответа



Регулярные задания у кандидата: задача, предложенная системой, и почему именно она



Проверка подхода: предварительная оценка по ключевым идеям и рубрика работодателя